ODACE CONSULTING
La synergie des expertises.

ABIDJAN · PARIS · CASABLANCA

DOSSIER D'ARCHITECTURE FONCTIONNELLE ET TECHNIQUE

Plateforme TestOps de JS2E

Orchestration des tests et des livraisons : ce que fait la solution, comment elle est construite et pourquoi

OBJET	Donner à JS2E une description complète et expliquée de la solution proposée par ODACE CONSULTING, pour validation en atelier de cadrage

Client : **JS2E** Établi par : **ODACE CONSULTING**

Version : **2.0 — 23 octobre 2026** Statut : **pour validation par JS2E**

Comment lire ce document

Chaque chapitre s'ouvre sur un encadré « En bref » qui explique l'idée en langage courant, avant les détails techniques. Un lecteur qui ne cherche que l'essentiel peut lire la synthèse, les encadrés et les schémas ; un lecteur technique poursuit avec les tableaux et les extraits.

PROFIL DE LECTEUR	PARCOURS CONSEILLÉ
Direction, sponsor	Synthèse, chapitres 2, 4, 14, 16
Responsables QA et qualité	Chapitres 2, 3, 4, 5, 8
Tech leads et développeurs	Chapitres 5, 6, 7, 8, 15
Exploitation, infrastructure	Chapitres 9, 11, 12, 13
Sécurité, conformité, audit	Chapitres 3, 7, 10, 11

Mentions utilisées

MENTION	SENS
Cahier des charges JS2E	Élément exprimé dans le cahier des charges TestOps et CI/CD du 23 octobre 2026.
Recommandation	Choix proposé par ODACE, à discuter en atelier.
Hypothèse	Point supposé faute d'information ; à confirmer au cadrage.
Option (lot CI/CD)	Capacité décrite pour montrer l'architecture cible complète ; traitée séparément dans l'offre.
Évolution	Hors première version ; prévue par l'architecture.

ODACE CONSULTING SARLU — Société à Responsabilité Limitée Unipersonnelle
Capital social : 10 000 000 FCFA · Société créée en 2022 · RCCM CI-ABJ-03-2022-B13-00315
N° Compte Contribuable 2208051 K · Numéro IDU CI-2022-0016887 M · N° employeur CNPS 399695
Siège social : Rue des Banques, 28 BP 628 Abidjan 28, Côte d'Ivoire
Bureau : 24 Rue Alphonse Daudet, face BGFI Bank, Plateau — Abidjan
Côte d'Ivoire : 05 05 13 12 06 · 07 08 96 96 09 — France : +33 6 03 77 08 93
contact@odaceconsulting.com · www.odaceconsulting.com — Abidjan (Côte d'Ivoire) · Paris (France) · Casablanca (Maroc)

S

Synthèse pour la direction

EN BREF JS2E veut un seul endroit pour lancer, suivre et prouver ses tests, puis faire avancer une version d'un environnement à l'autre en toute sécurité. La solution décrite ici le fait sans remplacer les outils que JS2E utilise déjà.

Ce que fait la solution

- Elle pilote les outils de test de JS2E (JMeter pour la charge, Selenium ou Playwright pour les parcours métier, scripts d'API) depuis une interface unique.
- Elle déclenche les tests à la demande, selon un calendrier ou à la fin d'un pipeline, en exécutant plusieurs campagnes en parallèle dans des conteneurs isolés.
- Elle calcule un verdict lisible : conforme, non conforme ou informatif, en tenant compte de la fiabilité de la mesure.
- Elle relie chaque version à ses tests, à son approbateur et à son déploiement, et garde les preuves exportables.
- Dans son lot CI/CD, elle orchestre la promotion entre environnements, l'approbation et le retour arrière.

Problèmes résolus

PROBLÈME	RÉPONSE
Tests lancés de façon dispersée, résultats non comparables	Catalogue unique de scripts versionnés, paramètres explicites, rapports consolidés
Risque de saturer une application de recette	Un test de charge à la fois par cible, quotas, plages horaires, arrêt d'urgence
Chiffres de performance faux, générateur saturé	Surveillance du générateur ; résultat marqué non fiable
Impossible de prouver qui a validé quoi	Journal d'audit, preuves exportables, relation commit → build → tests → déploiement
Mises en production sans critère objectif	Quality gates bloquants et approbation humaine

Principes de conception retenus

PRINCIPE (CAHIER DES CHARGES JS2E)	APPLICATION DANS L'ARCHITECTURE
Orchestration plutôt que remplacement des outils	Connecteurs et adaptateurs : JS2E garde ses outils et son pipeline
Traçabilité de chaque action	Journal d'audit en écriture seule ; historique des liens version-tests-déploiement
Architecture modulaire, API-first, observabilité native	Contrat de connecteur unique, API versionnée, métriques et traces dès le premier jour
Automatisation progressive, validations humaines aux étapes critiques	Verdict calculé distinct de la validation humaine ; approbation avant production

ODACE CONSULTING SARLU — Société à Responsabilité Limitée Unipersonnelle
Capital social : 10 000 000 FCFA · Société créée en 2022 · RCCM CI-ABJ-03-2022-B13-00315
N° Compte Contribuable 2208051 K · Numéro IDU CI-2022-0016887 M · N° employeur CNPS 399695
Siège social : Rue des Banques, 28 BP 628 Abidjan 28, Côte d'Ivoire
Bureau : 24 Rue Alphonse Daudet, face BGFI Bank, Plateau — Abidjan
Côte d'Ivoire : 05 05 13 12 06 · 07 08 96 96 09 — France : +33 6 03 77 08 93
contact@odaceconsulting.com · www.odaceconsulting.com — Abidjan (Côte d'Ivoire) · Paris (France) · Casablanca (Maroc)

Ce qui reste à valider avec JS2E

Les outils réellement en place, l'hébergement, le SSO, la volumétrie et les exigences de conservation ne sont pas connus à ce stade. Le chapitre 17 liste les points à trancher en atelier ; l'audit de découverte du lot 0 sert à les établir avant tout engagement ferme.

1

Introduction

EN BREF Ce chapitre dit à quoi sert le document, à qui il s'adresse et comment il est organisé.

Objet

Ce dossier décrit l'architecture de la plateforme proposée à JS2E : les fonctions offertes, les composants, les données, les intégrations, le déploiement, la sécurité, l'exploitation et le découpage en lots. Il répond au cahier des charges TestOps et CI/CD de JS2E (version 1.0 du 23 octobre 2026) et en reprend les principes, les rôles, les scénarios et les exigences, avec un tableau de correspondance au chapitre 16.

Méthode de représentation

Les vues vont du plus général au plus précis : contexte, fonctions, composants, données, déploiement. Un même vocabulaire est utilisé partout. Les schémas utilisent des couleurs constantes : bleu pour la plateforme, jaune pour l'exécution et les outils du client, vert pour les données et les résultats conformes, rouge pour les refus et les risques.

Documents associés

- Proposition technique d'ODACE (périmètre, critères de recette).
- Cahier des charges TestOps et CI/CD de JS2E, version 1.0 du 23 octobre 2026.
- Offre financière d'ODACE (document séparé).

2

Contexte et besoin de JS2E

EN BREF Avant de décrire la solution, ce chapitre reprend ce que JS2E a demandé et les questions qui restent ouvertes.

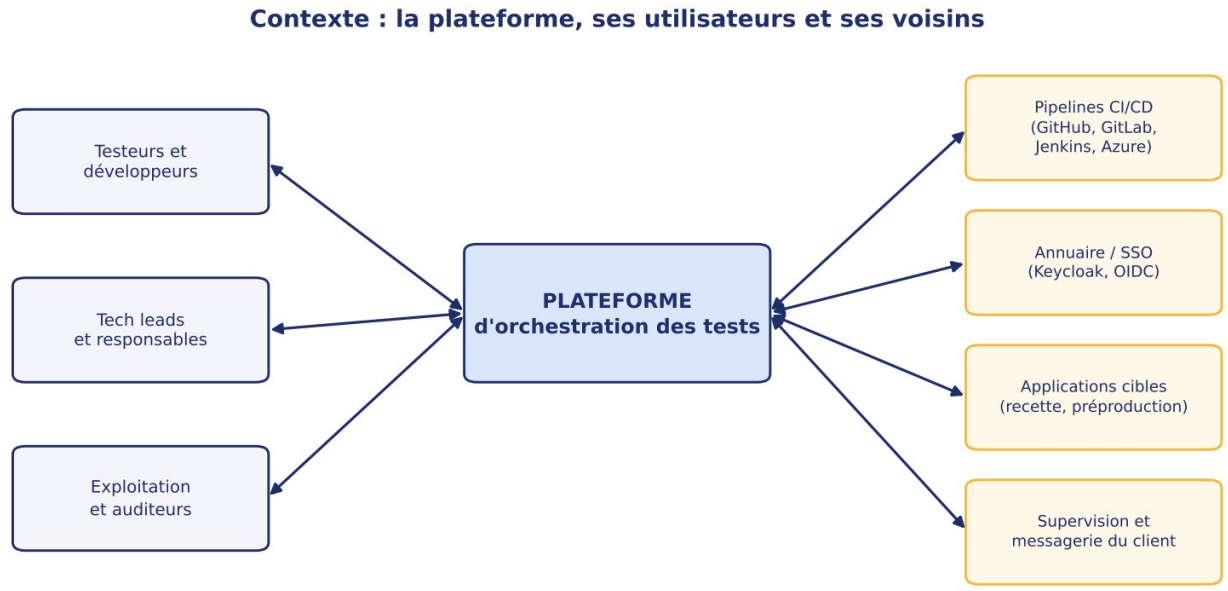


Figure 2.1. Contexte : la plateforme, ses utilisateurs et ses systèmes voisins.

Besoin exprimé

JS2E veut une plateforme centralisée d'orchestration des tests et des déploiements, intégrable à ses outils existants sans migration imposée. Les outils cités à titre d'exemple sont JMeter et Selenium ou Playwright. Les outils réellement en place ne sont pas connus : un inventaire préalable est obligatoire.

Périmètre prioritaire

CAPACITÉ DEMANDÉE PAR JS2E	OÙ ELLE EST TRAITÉE
Référentiel applications, environnements, équipes et scénarios	Chapitres 4, 6, 7
Import ou référencement de scénarios JMeter, Selenium/Playwright, tests API	Chapitre 6 (connecteurs)
Déclenchement manuel, planifié, sur webhook ou après pipeline	Chapitres 6, 8
Exécution parallèle avec quotas et isolation des runners	Chapitres 6, 10, 13
Rapports consolidés : succès, échec, durée, tendance, seuils, artefacts	Chapitres 4, 7
Orchestration CI/CD et promotion entre environnements	Chapitre 5 (lot CI/CD)

CAPACITÉ DEMANDÉE PAR JS2E	OÙ ELLE EST TRAITÉE
Quality gates bloquants ou informatifs configurables	Chapitre 4
Traçabilité et preuves d'exécution exportables	Chapitres 7, 10

Scénarios de référence de JS2E

SCÉNARIO	TRAITEMENT PAR LA PLATEFORME
Un QA choisit l'application et l'environnement, importe un plan JMeter, renseigne les paramètres et lance le test	Première version : parcours testeur (figure 4.3)
Le test s'exécute sur un runner autorisé ; courbes, erreurs et seuils s'affichent	Première version : agents isolés, suivi en direct
Après le build J2EE, les tests JUnit et API s'exécutent, puis le déploiement en recette a lieu	Tests : première version. Déploiement : lot CI/CD
Après le déploiement, une suite Selenium vérifie les parcours métier	Première version, déclenchée par webhook
Une campagne de performance non conforme bloque la promotion en production	Verdict transmis au pipeline : première version. Blocage de la promotion : lot CI/CD
Un responsable valide la release et suit l'état du déploiement	Validation des résultats : première version. Approbation de release et suivi du déploiement : lot CI/CD

Ce que la solution ne fait pas

- Elle ne remplace ni JMeter, ni Selenium, ni le pipeline de JS2E : elle les pilote.
- Elle ne décide pas de la mise en production : elle fournit un verdict et une approbation tracée, la décision reste humaine.
- Elle ne relance jamais seule un test de charge après un incident.
- Elle n'écrit pas les scripts de test à la place des équipes.
- Elle ne migre pas les données applicatives : les migrations irréversibles restent une décision humaine.

Hypothèses de travail

- Une application pilote, une version de JMeter, une version de Selenium ou Playwright ; infrastructure et accès fournis par JS2E.
- Hébergement sur site ou sur serveurs privés virtuels de JS2E, avec Docker.
- Aucun test de charge sur une cible non autorisée par écrit.

3

Exigences structurantes

EN BREF Une exigence est une promesse vérifiable. Ce chapitre liste les promesses fonctionnelles et de qualité, et dit comment chacune sera démontrée.

Exigences fonctionnelles

RÉF.	EXIGENCE	ORIGINE
EF1	Gérer applications, environnements, équipes, cibles autorisées et scripts versionnés	Cahier des charges JS2E
EF2	Lancer un test manuellement, selon un calendrier, sur webhook signé ou après un pipeline	Cahier des charges JS2E
EF3	Piloter JMeter, Selenium/Playwright et les tests d'API par des connecteurs	Cahier des charges JS2E
EF4	Exécuter en parallèle avec quotas, dans des runners isolés	Cahier des charges JS2E
EF5	Enchaîner des tests en workflows avec conditions	Cahier des charges JS2E
EF6	Calculer un verdict par seuils bloquants ou informatifs et le transmettre au pipeline	Cahier des charges JS2E
EF7	Fournir rapports, tendances, preuves exportables, notifications	Cahier des charges JS2E
EF8	Contrôler droits, quotas, plages horaires et arrêt d'urgence	Recommandation ODACE
EF9	Tracer toute action dans un journal d'audit	Cahier des charges JS2E
EF10	Orchestrer promotion, approbation, déploiement et retour arrière (lot CI/CD)	Cahier des charges JS2E

Exigences non fonctionnelles

RÉF.	DOMAINE	EXIGENCE	CIBLE (HYPOTHÈSE)
ENF1	Disponibilité	Plateforme accessible en heures ouvrées	99 % sur la plage de service
ENF2	Intégrité	Aucune exécution perdue sans trace ; aucun double lancement	0 double lancement aux tests de panne
ENF3	Performance de l'API	Actions courantes	95e centile inférieur à 500 ms hors export
ENF4	Capacité	Exécutions simultanées	À dimensionner au cadrage
ENF5	Sécurité	Moindre privilège, secrets hors journaux, MFA administrateurs	Test de fuite sans résultat
ENF6	Traçabilité	Conservation des journaux et preuves	Durée à définir avec JS2E
ENF7	Reprise	Restauration complète testée	Durée de reprise 8 h, perte maximale 24 h
ENF8	Maintenabilité	Versions d'images figées, migrations réversibles	Pipeline vert avant fusion
ENF9	Utilisabilité	Accessibilité clavier, adaptation à l'écran, filtres par rôle	Lancement et lecture du verdict en moins de 5 minutes

Critères d'acceptation métier de JS2E et preuve de recette

CRITÈRE DU CAHIER DES CHARGES	PREUVE EN RECETTE
Un utilisateur habilité lance un test depuis une seule interface	Démonstration par un QA sans aide, sur la cible de recette
Une exécution conserve sa version de scénario, ses paramètres non secrets et ses artefacts	Consultation d'une exécution ancienne : version, paramètres, artefacts présents
Le rapport distingue non exécutés, échecs, réussis et instables	Campagne mixte : comptes identiques au rapport source
Un quality gate refuse la promotion si le seuil est dépassé	Seuil volontairement dépassé : promotion bloquée et tracée (lot CI/CD)
Une livraison affiche commit, build, campagnes et approbateur	Fiche de release complète (lot CI/CD)
Un échec de déploiement permet un retour arrière documenté	Démonstration de retour arrière sur la recette (lot CI/CD)

Les cibles chiffrées sont des hypothèses de travail confirmées ou remplacées à l'atelier de cadrage.

4

Architecture fonctionnelle

EN BREF L'architecture fonctionnelle décrit ce que fait la solution, pour qui, et selon quelles règles, sans parler de technologie.

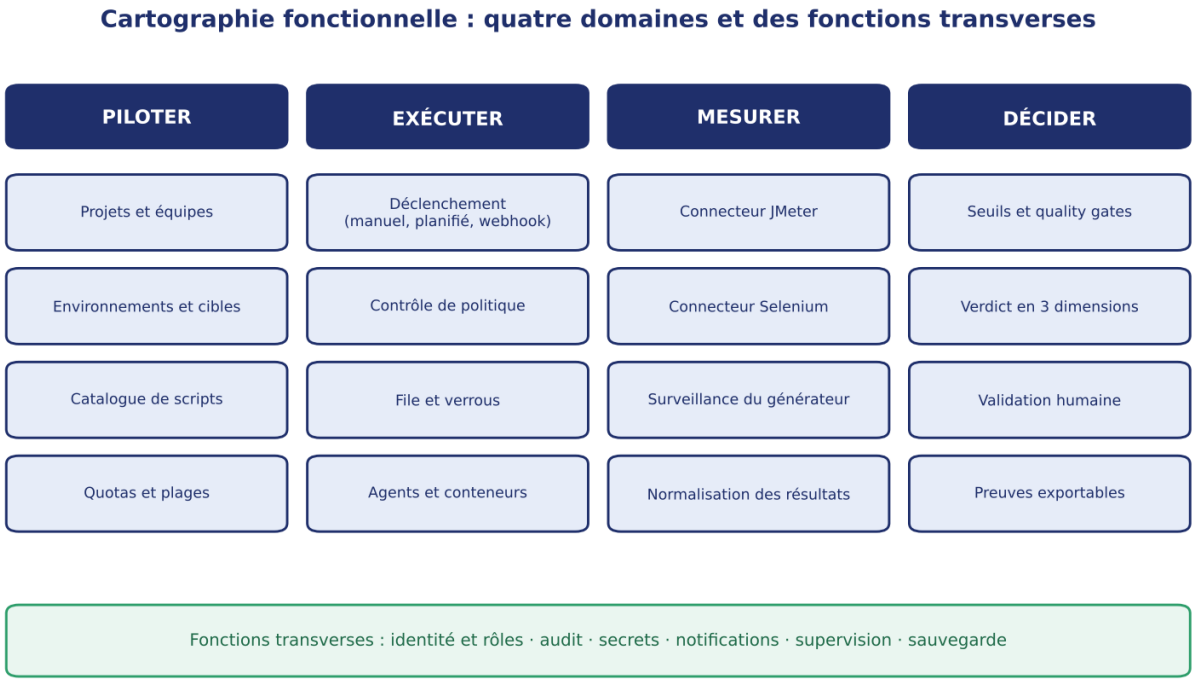


Figure 4.1. Cartographie fonctionnelle : quatre domaines et des fonctions transverses.

Quatre domaines se suivent : on pilote le référentiel (applications, environnements, scripts, quotas), on exécute (déclenchement, contrôles, file, agents), on mesure (connecteurs, surveillance, normalisation), puis on décide (seuils, verdict, validation, preuves). Le lot CI/CD ajoute un cinquième domaine, livrer, décrit au chapitre 5.

Acteurs et habilitations

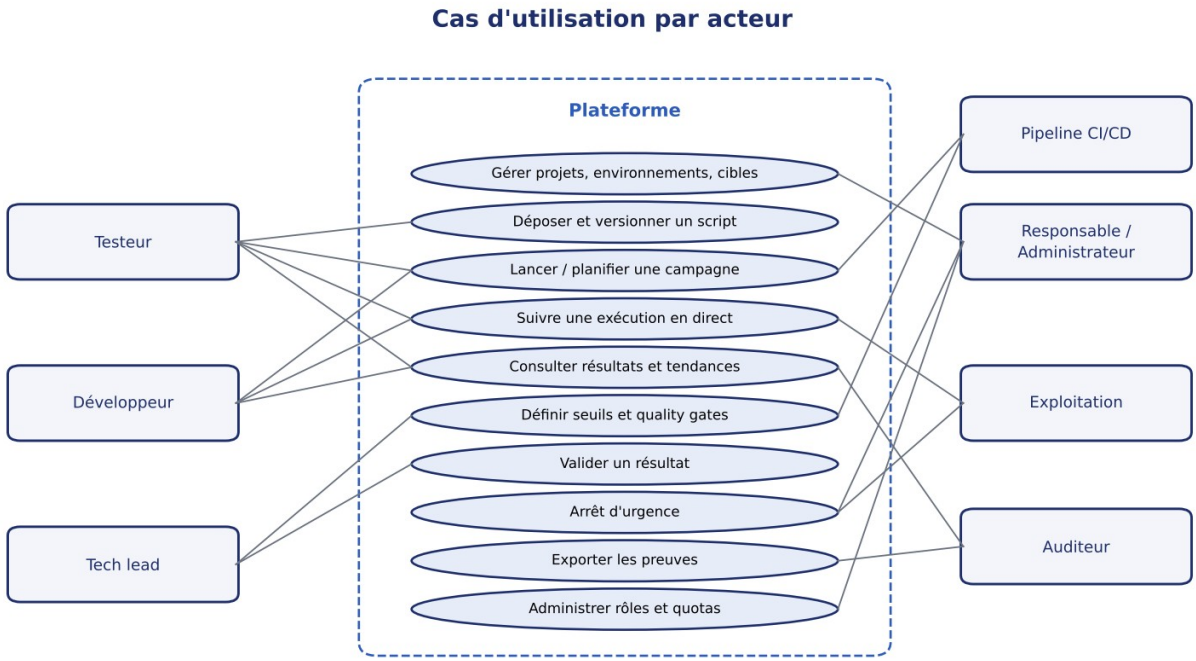


Figure 4.2. Cas d'utilisation par acteur.

ACTEUR (CAHIER DES CHARGES JS2E)	RESPONSABILITÉ DANS LA PLATEFORME
Administrateur plateforme	Paramétrage global, sécurité, connecteurs, arrêt d'urgence
Responsable entreprise	Équipes, projets, politiques et quotas
Tech lead	Validation technique, seuils, stratégie de livraison
Développeur	Consultation, déclenchement autorisé, journaux et corrections
QA	Campagnes, résultats, anomalies et qualité
Responsable exploitation	Infrastructure, incidents, déploiements, retour arrière
Auditeur	Lecture seule des preuves
Pipeline CI/CD (système externe)	Déclenche par webhook signé et lit le verdict

Parcours opérationnel de bout en bout

Le cahier des charges décrit dix étapes, de la création d'un espace projet au retour arrière. Le tableau montre ce qui est assuré dès la première version et ce qui relève du lot CI/CD.

ÉTAPE DU CAHIER DES CHARGES	QUI FAIT QUOI	COUVERTURE
1. Créer un espace projet avec rôles et politiques	Responsable entreprise, administrateur	Première version
2. Connecter le dépôt Git et vérifier les autorisations	Administrateur ; connecteur Git	Lot CI/CD

ÉTAPE DU CAHIER DES CHARGES	QUI FAIT QUOI	COUVERTURE
3. Enregistrer une application, son langage, son build, ses dépendances	Tech lead	Première version (fiche application) ; build en lot CI/CD
4. Déclarer les environnements et les cibles	Responsable exploitation	Première version
5. Sélectionner un modèle de pipeline versionné	Tech lead	Lot CI/CD
6. Configurer tests et seuils par environnement	Tech lead, QA	Première version
7. Déclencher sur pull request, merge, tag ou calendrier	Pipeline, planificateur	Webhooks et calendrier : première version ; liaison Git : lot CI/CD
8. Produire artefacts immuables, rapports et preuves	Agents, plateforme	Première version
9. Faire approuver la promotion en production	Approbateur	Lot CI/CD
10. Déployer, surveiller, revenir à la version précédente	Agents de déploiement	Lot CI/CD

Parcours d'un testeur, étape par étape

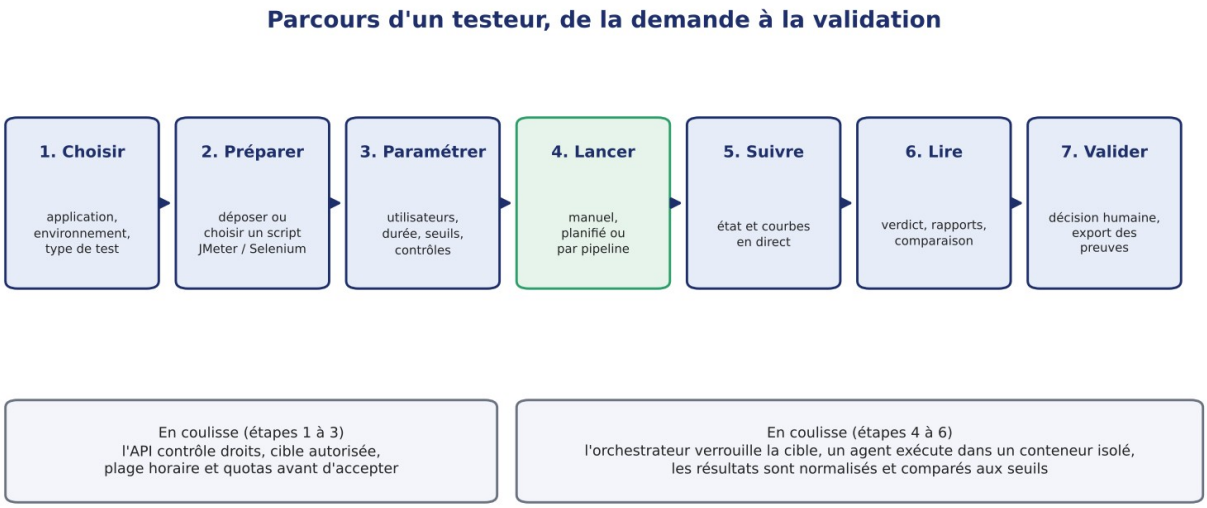


Figure 4.3. Du choix de l'application à la validation du résultat.

1. Choisir : le testeur sélectionne l'application, l'environnement et le type de test.
2. Préparer : il dépose un script ou choisit une version existante ; la plateforme calcule son empreinte pour garantir qu'on exécute bien ce qui a été validé.
3. Paramétrer : il renseigne les paramètres (nombre d'utilisateurs, durée, seuils). Les contrôles de politique s'affichent avant le lancement.
4. Lancer : manuellement, selon un calendrier, ou par un pipeline. Une même demande répétée ne crée qu'une exécution.
5. Suivre : état et courbes en direct.
6. Lire : verdict, rapports, comparaison avec les campagnes précédentes.
7. Valider : un responsable valide ou refuse ; les preuves sont exportables.

Cycle de vie d'une exécution

EN BREF Une exécution passe par des états précis. Savoir dans quel état elle se trouve, c'est savoir si on peut lui faire confiance, l'arrêter ou la relancer.

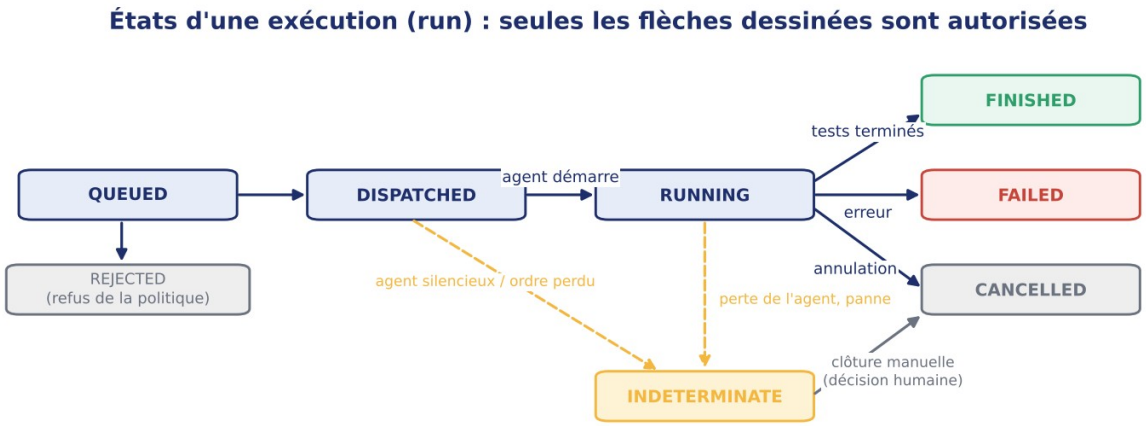


Figure 4.4. États d'une exécution et transitions autorisées.

ÉTAT	SIGNIFICATION	SUITE POSSIBLE
En file	Demande acceptée, en attente d'un agent libre	Envoyée, annulée, refusée
Envoyée	Ordre transmis à un agent	En cours, annulée, indéterminée
En cours	Le test s'exécute	Terminée, en erreur, annulée, indéterminée
Terminée	Le test est allé au bout ; le verdict est calculé à part	Aucune (état final)
En erreur	L'exécution a échoué techniquement	Aucune (état final)
Annulée	Arrêt demandé par un utilisateur ou par l'arrêt d'urgence	Aucune (état final)
Indéterminée	L'issue est inconnue (agent perdu, panne)	Clôture manuelle uniquement ; jamais de relance automatique

Le verdict en trois dimensions

EN BREF Un test « terminé » n'est pas forcément « bon ». Et un résultat « conforme » n'est pas forcément « fiable ». Nous séparons donc trois questions : le test s'est-il déroulé, le résultat respecte-t-il les seuils, et un humain l'a-t-il accepté.

DIMENSION	VALEURS	DÉCIDÉE PAR
État d'exécution	en file, envoyée, en cours, terminée, en erreur, annulée, indéterminée	Orchestrateur
Résultat	conforme, non conforme, informatif, non fiable	Calcul du quality gate
Validation	en attente, validée, refusée	Un humain (tech lead)

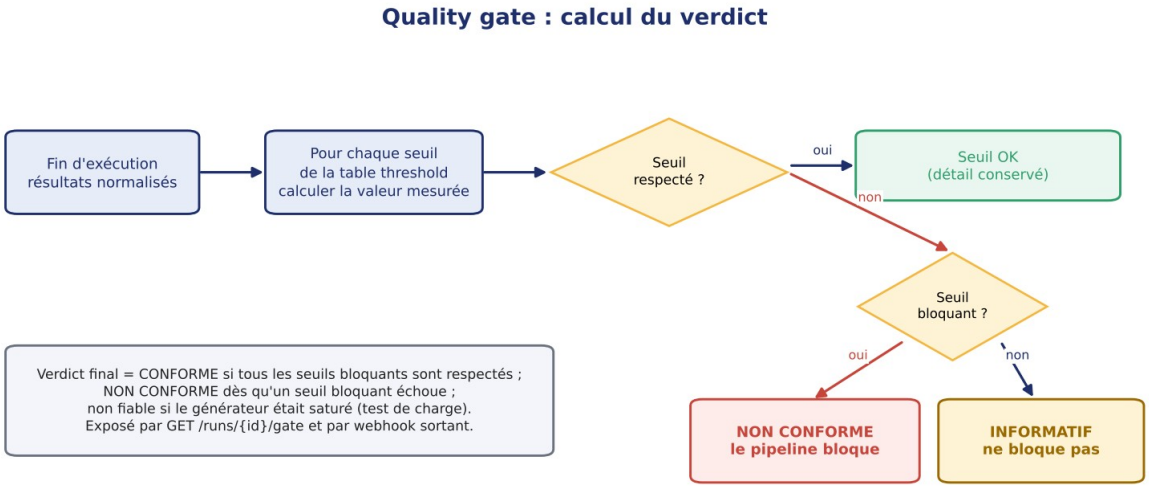


Figure 4.5. Calcul du verdict d'un quality gate.

Exemple : le seuil « temps de réponse au 95e centile inférieur ou égal à 800 ms » est bloquant et le seuil « nombre de tests instables inférieur ou égal à 2 » est informatif. Si le premier est dépassé, le verdict est non conforme et le pipeline s'arrête. Si seul le second est dépassé, le verdict reste conforme avec un avertissement. Si le générateur de charge était saturé, le résultat est marqué non fiable, quel que soit le chiffre mesuré.

Règles de gestion

RÉF.	RÈGLE	POURQUOI
RG1	Un seul test de charge à la fois sur une cible	Éviter de saturer l'application et de fausser les mesures
RG2	Une cible non autorisée pour ce type de test refuse la demande	Aucun test sans accord écrit
RG3	Une demande hors plage horaire est refusée	Protéger les fenêtres sensibles
RG4	Au-delà du quota, la demande attend en file	Maîtriser la charge de la plateforme
RG5	Un couple projet + clé d'idempotence ne crée qu'une exécution	Un appel répété par erreur ne double rien
RG6	Une exécution d'issue inconnue devient indéterminée, jamais relancée seule	Une relance automatique pourrait doubler la charge
RG7	Un test de charge n'est jamais rejoué seul ; un test Selenium en échec est rejoué une fois et marqué instable s'il réussit	Distinguer un défaut réel d'un aléa de navigateur
RG8	Un générateur saturé rend le résultat non fiable	Ne pas accuser l'application à tort
RG9	Seul un seuil déclaré bloquant produit un verdict non conforme	Éviter les blocages par surprise
RG10	La validation humaine est distincte du verdict calculé	Garder la décision à l'humain

RÉF.	RÈGLE	POURQUOI
RG11	Toute modification est écrite dans le journal d'audit	Preuve et traçabilité
RG12	L'arrêt d'urgence stoppe les nouvelles exécutions et ordonne l'arrêt des exécutions en cours	Réagir vite à un incident
RG13	Aucun déploiement en production sans approbation (à deux niveaux si exigé)	Lot CI/CD
RG14	Un retour arrière est tracé comme un déploiement	Lot CI/CD ; historique complet
RG15	Les migrations de données irréversibles ne sont jamais annulées automatiquement	Lot CI/CD ; décision humaine documentée

5

Module Livraison (lot CI/CD)

EN BREF Le cahier des charges de JS2E demande aussi d'orchestrer les déploiements. Ce chapitre décrit comment la plateforme le fait, en s'appuyant sur le pipeline que JS2E possède déjà. Ce module est traité comme un lot optionnel de l'offre ; il est décrit ici pour que JS2E voie l'architecture cible complète.

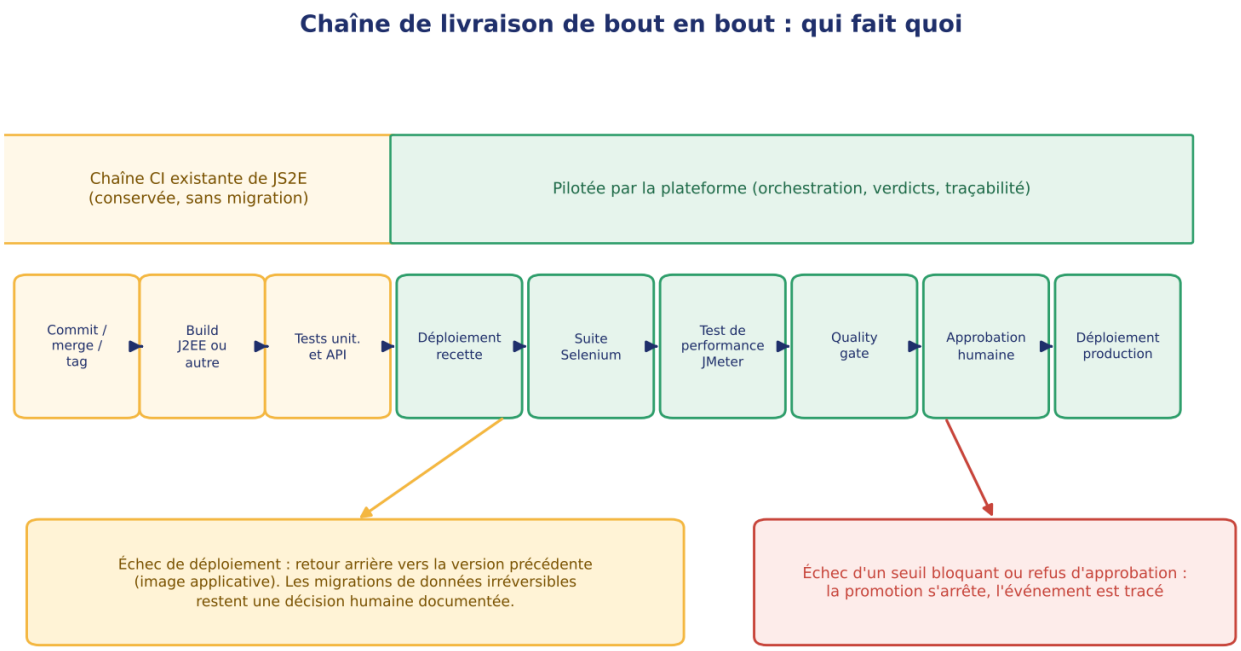


Figure 5.1. Chaîne de livraison de bout en bout : ce que garde JS2E et ce que pilote la plateforme.

Répartition des rôles

ÉTAPE	ASSURÉE PAR	RÔLE DE LA PLATEFORME
Commit, build, tests unitaires	Chaîne CI existante de JS2E	Reçoit l'événement ; relie le résultat à la release
Déploiement en recette	Agent de déploiement piloté par la plateforme	Ordonne, suit, enregistre la version déployée
Tests Selenium et API	Plateforme (connecteurs)	Exécute, normalise, calcule le verdict
Test de performance	Plateforme (connecteur JMeter)	Exécute, surveille le générateur, calcule le verdict
Quality gate	Plateforme	Autorise ou bloque la promotion
Approbation	Approbateur désigné	Demande, enregistre la décision et son auteur
Déploiement en production	Agent de déploiement	Ordonne, suit, prépare le retour arrière

Promotion entre environnements

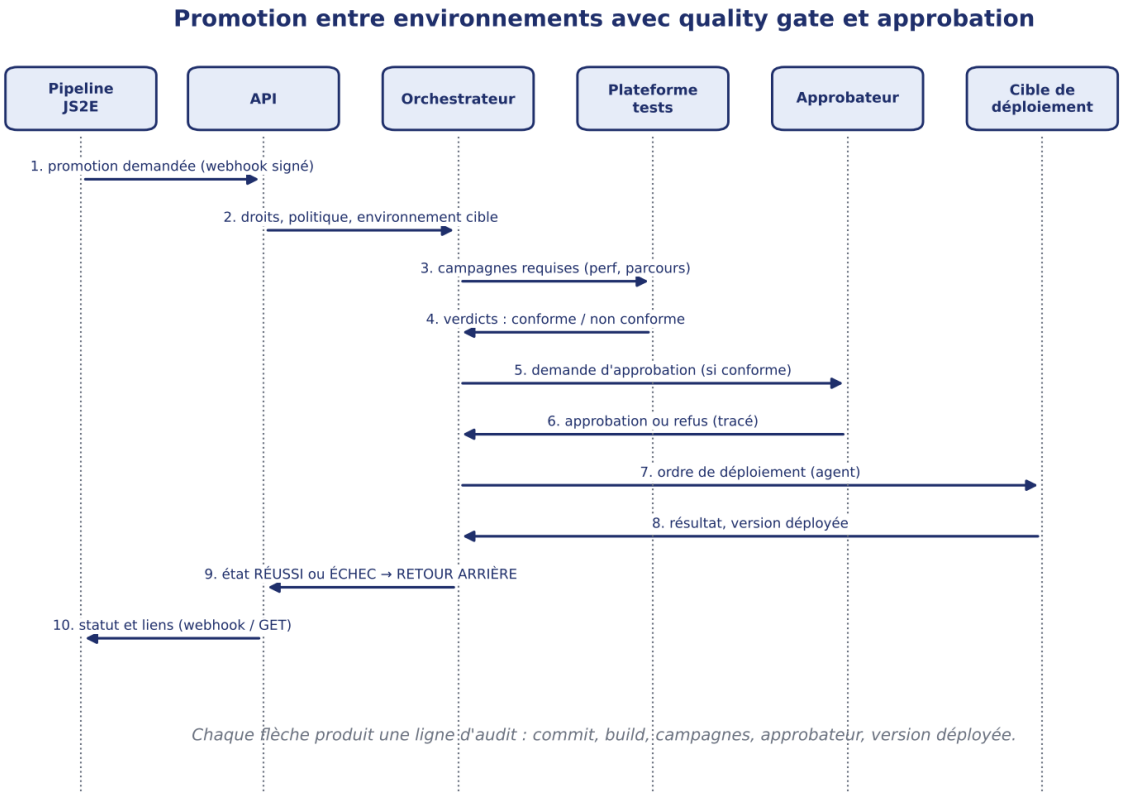
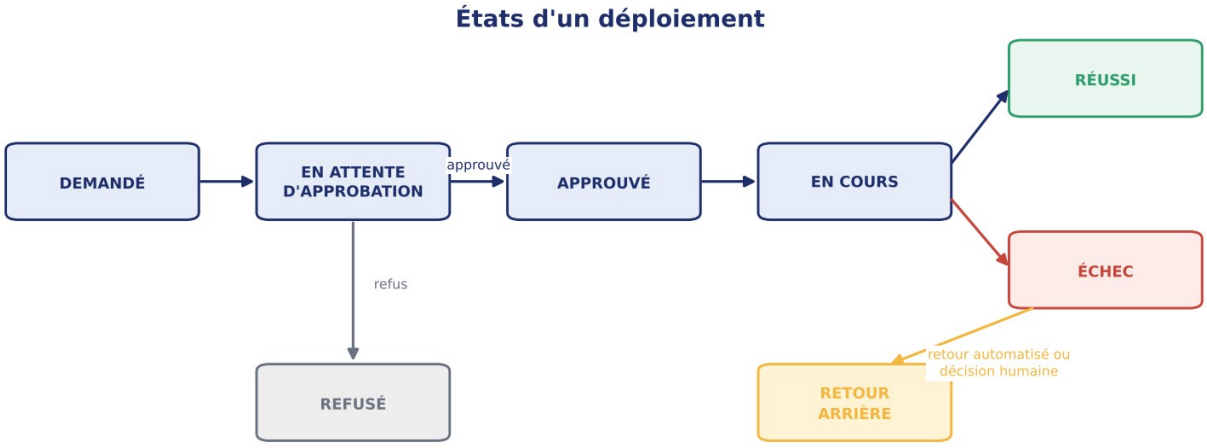


Figure 5.2. Promotion avec quality gate et approbation.

1. Le pipeline de JS2E demande la promotion d'une release vers l'environnement suivant, par webhook signé.
2. L'API vérifie les droits, la politique de l'environnement cible et la plage horaire.
3. L'orchestrateur lance les campagnes exigées (performance, parcours) sur l'environnement source.
4. Les verdicts reviennent. Un seuil bloquant non respecté arrête la promotion et la trace.
5. Si tout est conforme, une demande d'approbation est envoyée à l'approbateur, à deux niveaux si JS2E l'exige.
6. L'approbation ou le refus est enregistré avec son auteur et l'heure.
7. En cas d'approbation, l'orchestrateur ordonne le déploiement à l'agent de la cible.
8. L'agent rend compte ; la plateforme enregistre la version déployée, ses liens avec le commit et le build.
9. En cas d'échec, le retour arrière est proposé ou lancé selon la politique de l'environnement.
10. Le statut et les liens sont mis à disposition du pipeline par webhook ou par appel.

États d'un déploiement



Aucun déploiement en production sans approbation (à deux niveaux si exigé). Le retour arrière est tracé comme un déploiement à part entière.

Figure 5.3. États d'un déploiement et du retour arrière.

Retour arrière : ce qui est automatisable et ce qui ne l'est pas

EN BREF Revenir à la version précédente d'une application, c'est simple si seul le code change. C'est dangereux si une base de données a déjà été transformée. Nous séparons donc les deux cas.

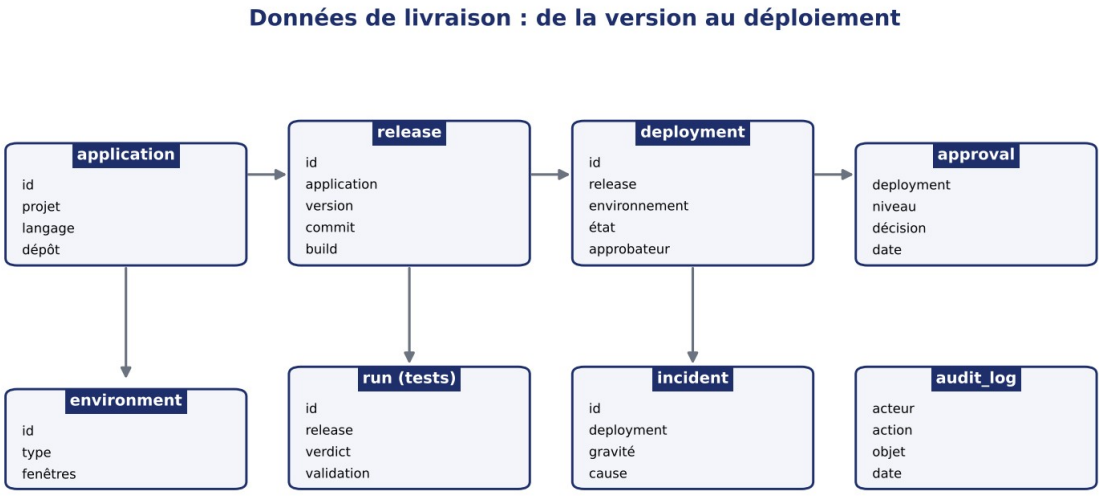
CAS	TRAITEMENT
Retour de l'image applicative (conteneur, paquet)	Automatisable : la version précédente reste disponible et est redéployée
Configuration	Automatisable si elle est versionnée avec la release
Migration de données réversible	Script inverse fourni et testé en recette
Migration de données irréversible	Jamais annulée automatiquement ; décision humaine, plan documenté et validé avant la mise en production

Cibles de déploiement

CIBLE (CAHIER DES CHARGES)	MÉCANISME PROPOSÉ	À VALIDER
Serveurs privés virtuels, machines	Agent installé ou connexion SSH à identité de courte durée	Accès réseau, comptes de service
Docker	Mise à jour de conteneurs par l'agent	Registre d'images, politique de versions
Kubernetes	Application de manifestes versionnés	Existence et gouvernance d'un cluster

CIBLE (CAHIER DES CHARGES)	MÉCANISME PROPOSÉ	À VALIDER
Autres	Via connecteur spécifique	Selon l'inventaire

Traçabilité d'une livraison



Chaîne historisée : commit → build → release → campagnes de test → déploiement → incident. Chaque lien est horodaté.

Figure 5.4. Données de livraison : de la version au déploiement.

Une fiche de release affiche en une page : le commit, le build, les campagnes de test et leurs verdicts, l'approbateur, l'heure et la version déployée, ainsi que les incidents éventuels. C'est la réponse au critère de JS2E « une livraison affiche le commit, le build, les campagnes et l'approbateur ».

6

Architecture applicative

EN BREF Les composants sont les « pièces » de la solution. Chacun a une seule responsabilité claire, ce qui permet de les faire évoluer ou de les remplacer sans tout casser.

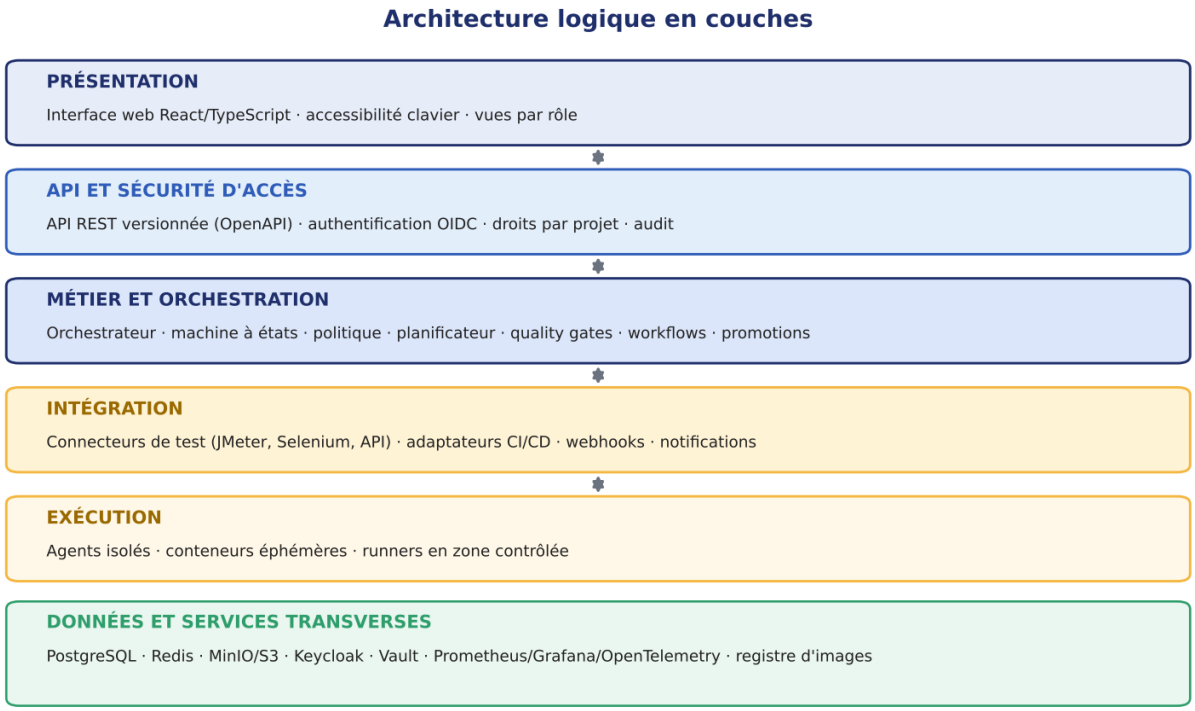
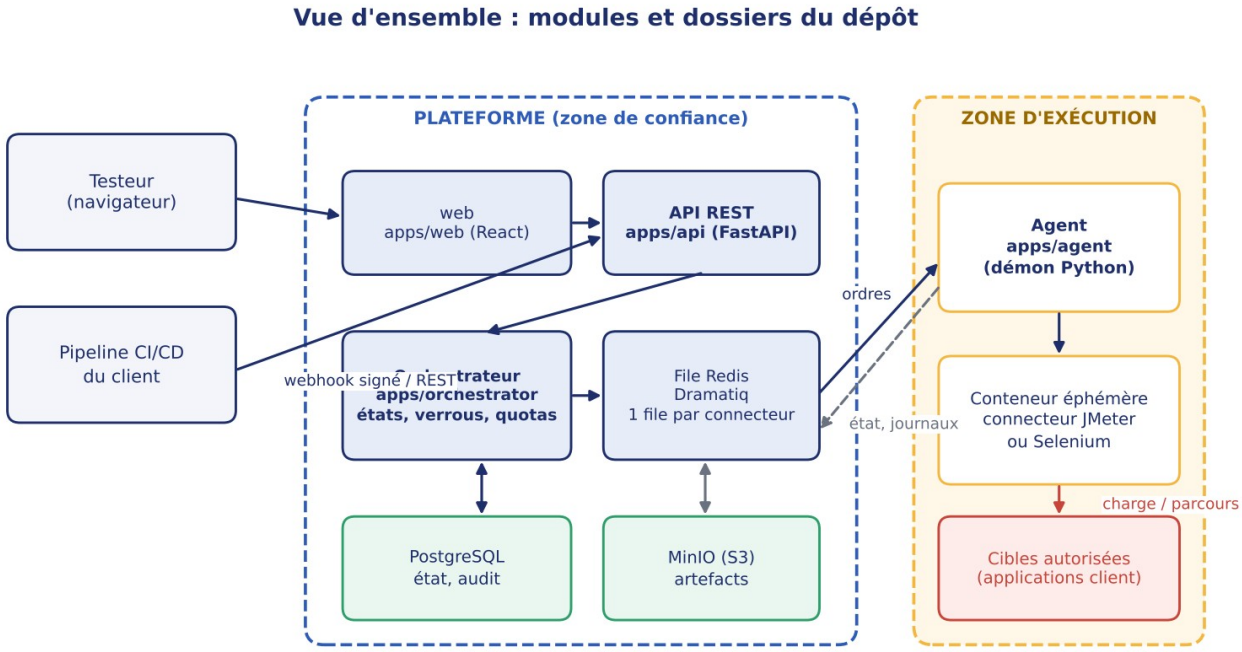


Figure 6.1. Architecture logique en couches.



Les agents n'ouvrent aucun port entrant : ils interrogent l'orchestrateur (flux sortant uniquement).

Figure 6.2. Composants, zones de confiance et organisation du code.

Composants et responsabilités

COMPOSANT	RESPONSABILITÉ	CE QU'IL NE FAIT PAS	TECHNOLOGIE
Interface web	Écrans, suivi en direct, administration	Aucune règle métier ; ne calcule pas de verdict	React, TypeScript
API REST	Point d'entrée unique, droits, validation, audit	N'exécute pas de test	Python 3.12, FastAPI
Orchestrateur	États, politique, verrous, quotas, surveillance	N'interprète pas les résultats des moteurs	Python
Planificateur	Déclenchements planifiés, interdiction de chevauchement	-	Dans l'orchestrateur
File de tâches	Transmet les ordres aux agents	Ne stocke pas d'état de vérité	Redis, Dramatiq
Agent d'exécution	Lance les conteneurs, remonte état et artefacts	N'ouvre aucun port entrant	Python, Docker
Connecteurs de test	Valident, préparent, lisent les résultats d'un outil	Ne décident pas du verdict final	Python + outil
Adaptateurs CI/CD	Parlent à GitHub Actions, GitLab CI, Jenkins, Azure DevOps	Ne remplacent pas les pipelines	Python (lot CI/CD)

COMPOSANT	RESPONSABILITÉ	CE QU'IL NE FAIT PAS	TECHNOLOGIE
Base de données	État, configuration, audit	-	PostgreSQL 16
Stockage d'artefacts	Scripts, rapports, captures	-	MinIO (compatible S3)
Identité	Authentification, groupes, rôles	-	Keycloak (OIDC)
Coffre de secrets	Secrets de cibles et d'intégrations	Aucun secret n'est copié en base	Vault ou équivalent JS2E
Supervision	Métriques, journaux, traces, alertes	-	Prometheus, Grafana, OpenTelemetry

Séquence nominale d'une exécution

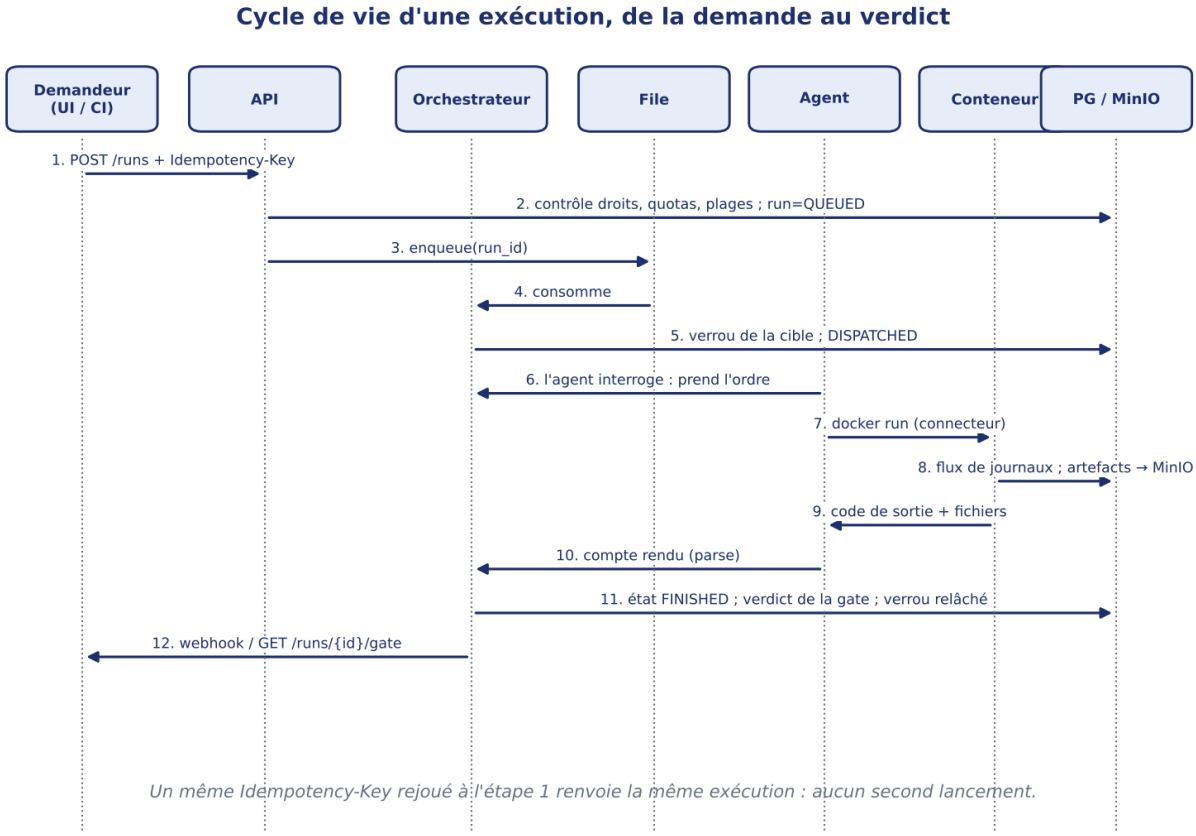


Figure 6.3. De la demande au verdict : échanges entre composants.

Contrôles avant l'envoi à un agent

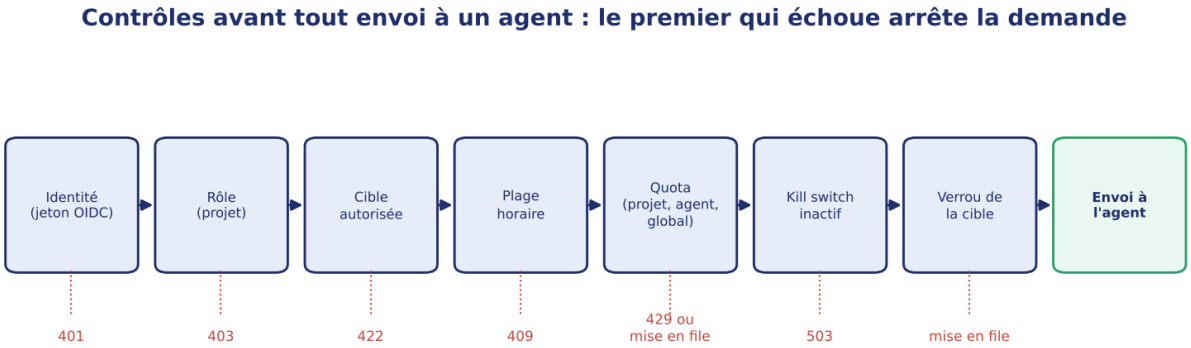


Figure 6.4. Chaîne de contrôles ; le premier qui échoue arrête la demande.

Le contrat de connecteur

EN BREF Un connecteur est un adaptateur entre la plateforme et un outil de test. Tous suivent le même contrat de cinq opérations : ajouter un nouvel outil revient à écrire un connecteur, sans modifier le cœur.

```
class Connector(Protocol):
    kind: str
    def validate(self, spec) -> list[Problem]: ...    # avant tout lancement
    def prepare(self, spec, workdir) -> Command: ...  # ligne de commande, environnement
    def parse(self, workdir) -> Result: ...           # métriques et statuts normalisés
    def artifacts(self, workdir) -> list[Path]: ...
    def stop(self, handle, grace_s) -> None: ...
```

Connecteur JMeter

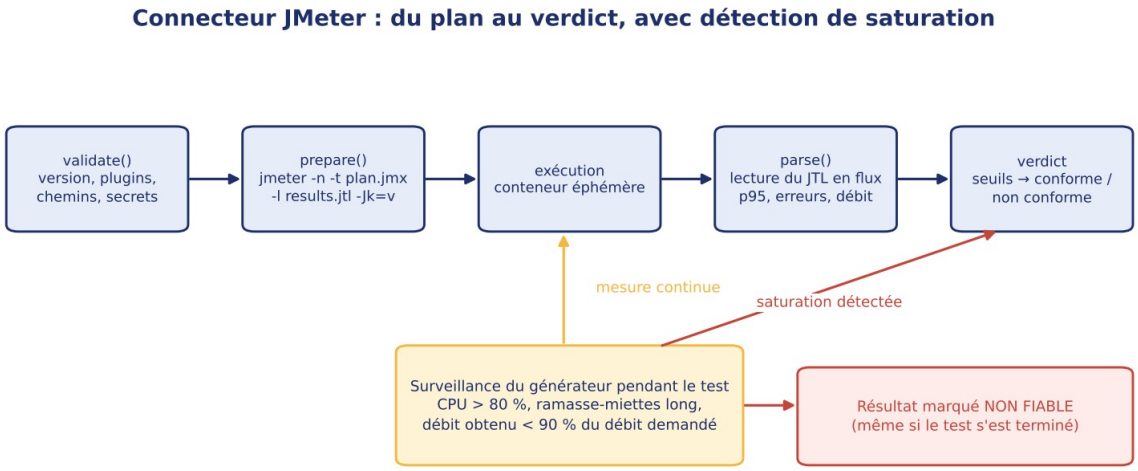


Figure 6.5. JMeter : du plan au verdict, avec détection de saturation.

ASPECT	DÉTAIL
Entrée	Fichier JMX versionné, paramètres passés en ligne de commande, jamais écrits dans le plan
Validation	Version compatible, plugins présents, aucun chemin absolu, aucun secret en clair, nombre d'utilisateurs inférieur au maximum de la cible
Exécution	Mode non graphique uniquement, dans un conteneur éphémère
Sortie	Fichier JTL et rapport HTML ; lecture en flux ; débit, temps de réponse (moyenne, 90e, 95e, 99e centile), taux d'erreur
Saturation	Processeur élevé, ramasse-miettes long ou débit obtenu inférieur à 90 % du débit visé : résultat marqué non fiable
Charge répartie	Plusieurs générateurs possibles, démarrage synchronisé, agrégation des résultats

Connecteur Selenium et Playwright

ASPECT	DÉTAIL
Entrée	Suite versionnée ; navigateur et pilote à version précise dans l'image
Sortie	Rapport JUnit XML, captures d'écran, journal de console, trace réseau, nommés par test
Statuts normalisés	Réussi, en échec, non exécuté, instable
Nouvelle tentative	Une seule, sur échec ; un test réussi à la seconde tentative est marqué instable
Garde-fou	Aucune option de montée en charge : Selenium vérifie des parcours, il ne mesure pas la performance sous charge
Playwright	Même contrat ; ajouté après qualification avec JS2E

Connecteur de tests d'API

Collections ou scripts exécutés dans un conteneur dédié, avec export de résultats normalisé dans le même format que les autres connecteurs. L'outil exact (par exemple une collection exportée ou un script Python) est choisi après l'inventaire.

Workflows

Un workflow enchaîne des étapes avec dépendances et conditions. Il est décrit dans un fichier lisible, versionné avec le projet.

```
name: recette-appli-x
steps:
  - id: perf
    kind: jmeter
    asset: plan-charge-v3
    target: recette-appli-x
    params: {threads: 200, duration: 600}
  - id: parcours
    kind: selenium
    asset: suite-parcours-v5
    target: recette-appli-x
    needs: [perf]
    when: perf.result != "non_conforme"
gate:
  blocking: [perf.p95_ms <= 800, perf.error_rate <= 0.01]
  informative: [parcours.flaky_count <= 2]
```

API et événements

ROUTE (PREMIÈRE VERSION)	RÔLE MINIMAL	USAGE
POST /runs	Développeur autorisé	Demander un lancement (clé d'idempotence obligatoire)
POST /runs/{id}/cancel	QA	Annuler
POST /runs/{id}/validate	Tech lead	Validation humaine
GET /runs/{id}, /events, /artifacts	Lecture	Suivi, résultats, preuves
GET /runs/{id}/gate	Lecture ou pipeline	Verdict du quality gate, attente possible
POST /assets	QA	Déposer un script, empreinte et version
PUT /thresholds	Tech lead	Seuils bloquants ou informatifs
POST /hooks/{id}	Signature HMAC	Déclenchement par webhook signé
GET /audit-events	Auditeur	Consulter le journal
POST /admin/kill	Administrateur, exploitation	Arrêt d'urgence

ROUTE CITÉE PAR JS2E	CORRESPONDANCE PROPOSÉE
POST /projects ; GET /projects	Identique
POST /connectors/github/verify	Lot CI/CD : vérification du connecteur Git
POST /pipelines/{id}/runs ; GET /runs/{id}/logs	Lot CI/CD : exécution d'un pipeline et journaux

ROUTE CITÉE PAR JS2E	CORRESPONDANCE PROPOSÉE
POST /test-campaigns/{id}/execute ; GET /test-runs/{id}/results	Équivalent de POST /runs et GET /runs/{id} ; nommage à aligner en atelier
POST /deployments ; /approve ; /rollback	Lot CI/CD : même nommage
GET /audit-events ; GET /metrics/summary	Identique

Événements publiés (webhooks sortants) : source.pushed, pipeline.started, tests.failed, release.approved, deployment.completed, auxquels s'ajoutent run.finished, run.indeterminate et gate.evaluated. Conventions : JSON, pagination par curseur, schéma OpenAPI publié, format d'erreur unique avec identifiant de corrélation, horodatages en UTC, clé d'idempotence sur les actions de lancement.

7

Architecture des données

EN BREF Les données sont le cœur de la preuve : sans elles, impossible de dire qui a lancé quoi, avec quelle version, et ce que cela a donné.

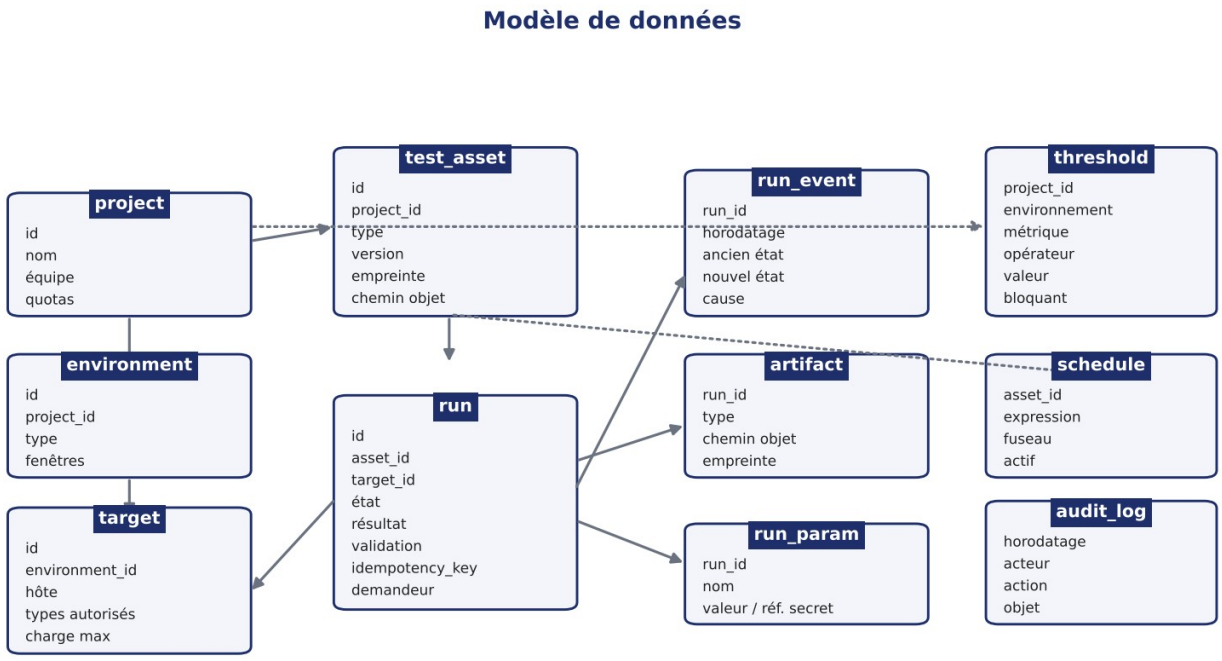


Figure 7.1. Modèle de données des tests : douze tables.

Dictionnaire des données principales

DONNÉE	CONTENU	STOCKAGE	SENSIBILITÉ
Référentiel	Organisation, équipes, projets, applications, environnements, cibles, quotas	PostgreSQL	Interne
Scripts	Plans JMeter, suites Selenium, collections d'API, versions et empreintes	MinIO + index PostgreSQL	Interne ; peut contenir des adresses
Exécutions	État, résultat, validation, paramètres non secrets	PostgreSQL	Interne
Événements	Changements d'état horodatés	PostgreSQL (écriture seule)	Interne
Artefacts	Rapports natifs, captures, journaux	MinIO	Peut contenir des données applicatives

DONNÉE	CONTENU	STOCKAGE	SENSIBILITÉ
Secrets	Identifiants de cibles et d'intégrations	Coffre ; seules des références en base	Confidentiel
Livraison	Releases, déploiements, approbations, incidents	PostgreSQL	Interne (lot CI/CD)
Audit	Qui a fait quoi, quand, depuis où	PostgreSQL (écriture seule)	Confidentiel

Le cahier des charges de JS2E évoque PostgreSQL ou Supabase avec politiques de sécurité par ligne. L'architecture proposée utilise PostgreSQL ; les politiques par ligne y sont possibles et peuvent doubler les contrôles côté serveur si JS2E le demande. Le filtrage côté serveur, lui, est systématique dès la première version.

Historisation des relations

Les liens entre commit, version, tests, déploiement et incident sont conservés avec leur date. Ils permettent de répondre à des questions comme : quelle version tournait en production à telle date, quels tests l'avaient validée, qui l'avait approuvée, quel incident a suivi.

Cycle de vie et conservation

DONNÉE	RÈGLE PROPOSÉE (HYPOTHÈSE)
Résultats résumés et événements	Conservés pour les tendances ; durée à fixer avec JS2E
Artefacts volumineux (JTL, captures)	Conservation plus courte, puis suppression tracée
Audit	Conservé selon la politique de sécurité de JS2E ; jamais modifié
Preuves exportées	Archive avec empreintes, versionnée

Volumétrie indicative (hypothèse)

ÉLÉMENT	ORDRE DE GRANDEUR	REMARQUE
Exécutions par jour	Quelques dizaines	À confirmer avec JS2E
Fichier JTL d'un test d'une heure	Plusieurs centaines de mégaoctets	Lu en flux, jamais chargé en mémoire
Événements par exécution	Une dizaine	Faible volume
Captures d'un test en échec	Quelques mégaoctets	Conservation limitée

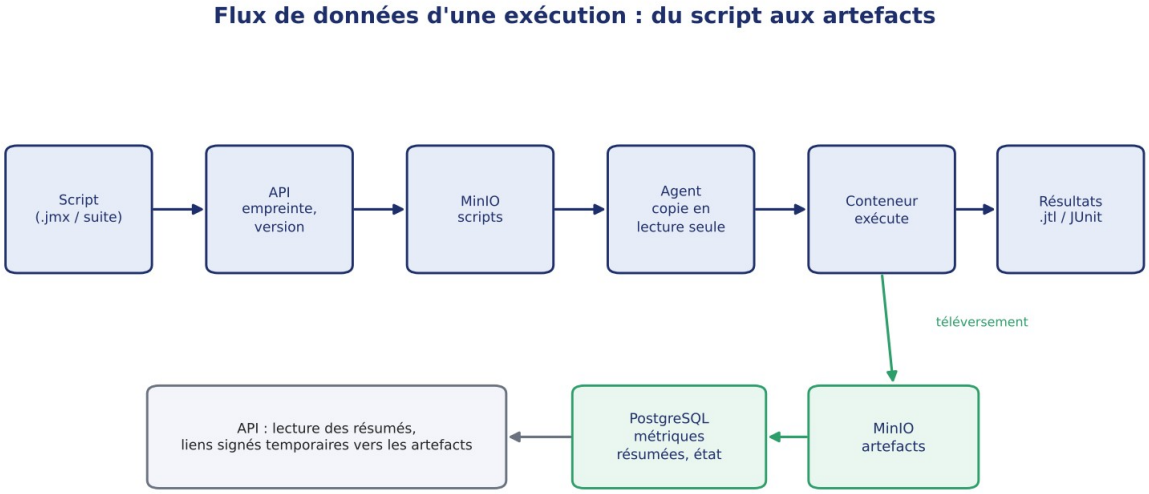


Figure 7.2. Flux de données d'une exécution : du script aux artefacts.

8

Architecture d'intégration

EN BREF La plateforme n'est utile que si elle se branche sur ce que JS2E possède déjà : son pipeline, son annuaire, sa messagerie. Ce chapitre décrit ces branchements.

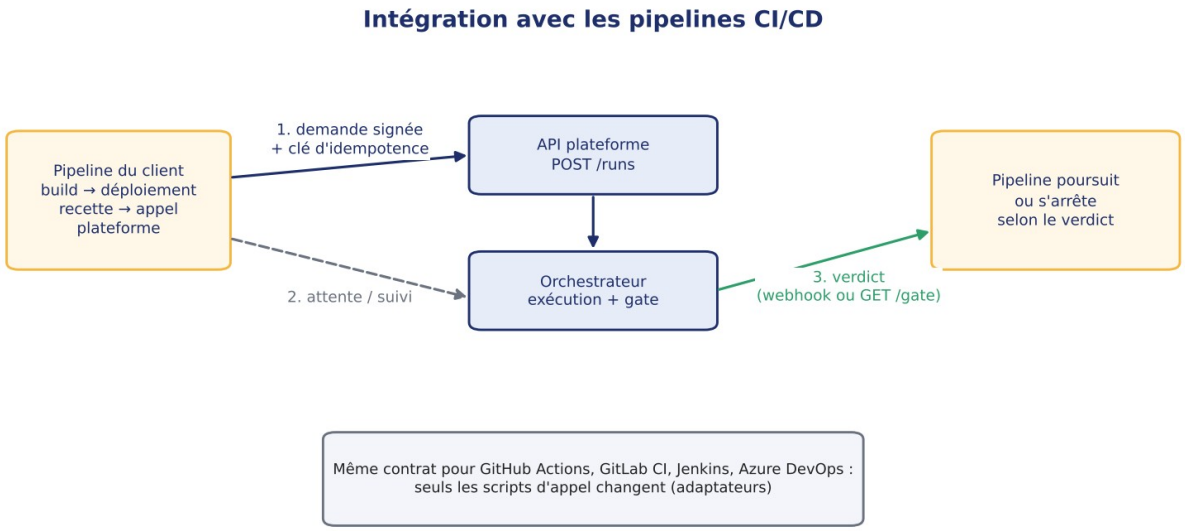


Figure 8.1. Le pipeline de JS2E appelle la plateforme et lit son verdict.

Mécanismes d'échange

MÉCANISME	SENS	DÉTAIL
Webhook entrant signé	JS2E vers plateforme	HMAC-SHA256, horodatage maximal de cinq minutes, identifiant d'événement
API REST	JS2E vers plateforme	Jeton d'un compte de service, clé d'idempotence obligatoire
Verdict	Plateforme vers JS2E	GET /runs/{id}/gate avec attente, ou webhook sortant signé
Notifications	Plateforme vers JS2E	Courriel et webhook ; Teams ou Slack selon intégrations validées
Formats	Les deux sens	JSON, JUnit XML ; métriques Prometheus si disponibles

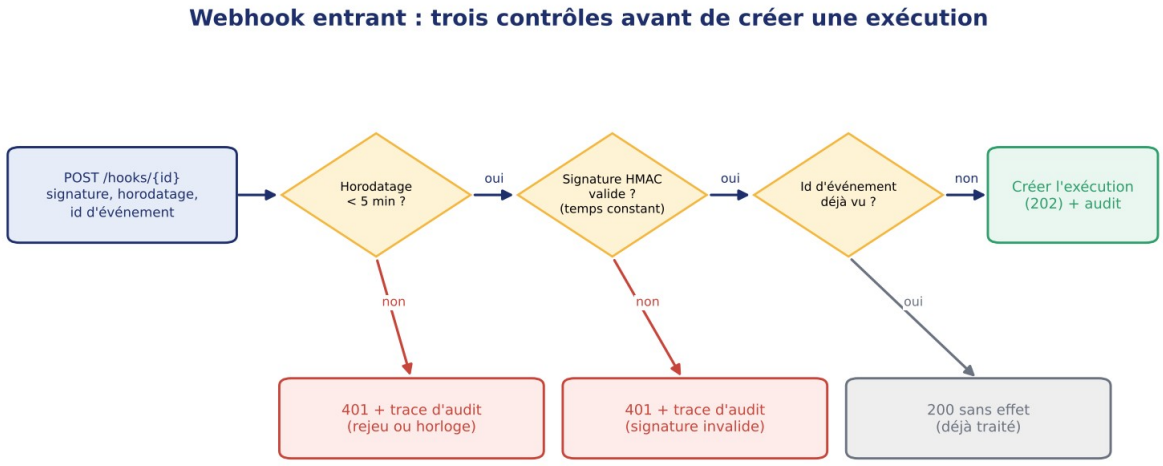


Figure 8.2. Les trois contrôles d'un webhook entrant.

EN BREF Un webhook est un appel automatique d'un système à un autre. Pour éviter qu'un inconnu déclenche des tests, chaque appel est signé avec un secret partagé, daté, et reconnu une seule fois.

Adaptateurs CI/CD

OUTIL	DÉCLENCHEMENT PAR L'OUTIL	LECTURE DU VERDICT	POINT D'ATTENTION
GitHub Actions	Étape de workflow : appel REST	Étape avec attente du verdict	Secrets du dépôt, permissions du jeton
GitLab CI	Job : appel REST	Job avec attente du verdict	Variables protégées, runners
Jenkins	Étape de Jenkinsfile : appel REST	Étape avec attente	Gestion des identifiants Jenkins, plugins
Azure DevOps	Tâche de pipeline : appel REST	Tâche avec attente	Connexions de service

Le contrat d'appel est identique pour les quatre outils ; seul le petit script d'appel change. Chaque adaptateur est qualifié sur l'outil réellement utilisé par JS2E avant d'être promis.

Identité

- OpenID Connect avec Keycloak ; fédération possible vers l'annuaire de JS2E.
- Rôles portés par des groupes ; correspondance groupes-rôles configurable.
- Authentification multifacteur obligatoire pour les administrateurs.
- Comptes de service dédiés aux pipelines, jetons à durée limitée.

Autres intégrations

OUTIL	USAGE	ÉTAT
Gestionnaire de tickets	Création d'anomalies depuis un échec	Évolution, selon l'outil de JS2E
Teams, Slack, courriel	Notifications	Selon intégrations validées
Supervision existante	Réutilisation de Prometheus ou Grafana	Selon l'existant
Registre d'images OCI	Images d'exécution versionnées	Registre de JS2E ou fourni

9

Architecture technique et déploiement

EN BREF Ce chapitre dit où les composants s'installent, comment ils communiquent et ce qu'il faut prévoir côté serveurs et réseau.

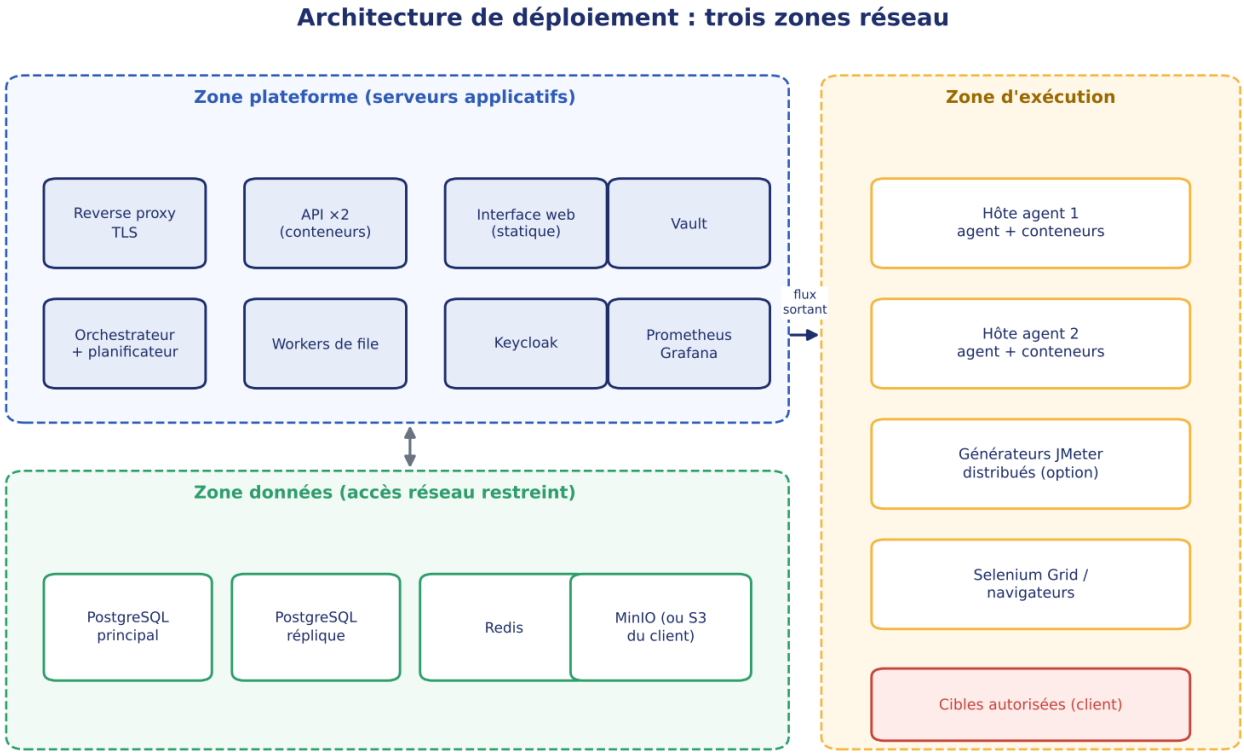


Figure 9.1. Déploiement en trois zones réseau.

Zones et flux autorisés

SOURCE	DESTINATION	PROTOCOLE (INDICATIF)	MOTIF
Utilisateurs	Reverse proxy	HTTPS 443	Interface et API
Pipelines JS2E	Reverse proxy	HTTPS 443	Webhooks et API
API, orchestrateur	PostgreSQL	5432	État, audit
API, orchestrateur	Redis	6379	File de tâches
API, agents	MinIO	9000 (HTTPS)	Scripts et artefacts
Agents	Orchestrateur	HTTPS, flux sortant	Ordres, état, battement de cœur
Agents	Cibles autorisées	Selon la cible	Exécution des tests

SOURCE	DESTINATION	PROTOCOLE (INDICATIF)	MOTIF
Plateforme	Annuaire / SSO	HTTPS ou LDAPS	Identité
Plateforme	Messagerie, webhooks JS2E	HTTPS, SMTP	Notifications

Aucun port n'est ouvert vers les agents : ils interrogent l'orchestrateur. Les ports sont indicatifs et dépendent des règles de sécurité de JS2E.

Environnements

ENVIRONNEMENT	USAGE	PARTICULARITÉS
Développement	Un par développeur (conteneurs locaux)	Cible factice, données de démonstration
Intégration	Tests automatisés du pipeline de la plateforme	Services réels en conteneurs, éphémère
Recette	Démonstration et recette par JS2E	Configuration proche de la production, sauvegarde testée
Production	Usage réel	Accès restreint, supervision et sauvegardes actives

Dimensionnement indicatif (hypothèse)

ÉLÉMENT	POINT DE DÉPART	À AJUSTER SELON
Serveur plateforme	4 vCPU, 16 Go de mémoire, 100 Go de disque	Nombre d'utilisateurs et d'exécutions
PostgreSQL	2 vCPU, 8 Go, 200 Go	Rétention des événements
Stockage objet	Capacité selon la durée de conservation	Volume des JTL et captures
Hôte d'agent de charge	8 vCPU, 16 Go par agent	Charge visée ; le générateur est le goulot
Hôte de navigateurs	4 vCPU, 16 Go pour quelques navigateurs	Suites simultanées

Ces valeurs sont des points de départ, validés par un essai de charge du générateur sur l'application pilote pendant le cadrage.

Livraison et configuration

- Images conteneur versionnées et analysées, construites par le pipeline ; aucune version « latest ».
- Infrastructure décrite par des fichiers versionnés (Infrastructure as Code) et scripts d'installation, livrés à JS2E.
- Configuration par variables d'environnement ; secrets exclusivement dans le coffre.
- Déploiement par docker-compose en recette et production sur site ; un orchestrateur de conteneurs plus complet est une évolution si JS2E en dispose.
- Migrations de base appliquées avant le démarrage des nouvelles versions, réversibles.

10

Sécurité

EN BREF Une plateforme qui lance des tests sur des applications réelles est une cible de valeur. La sécurité repose sur trois idées : donner le moins de droits possible, isoler chaque exécution, tout tracer.

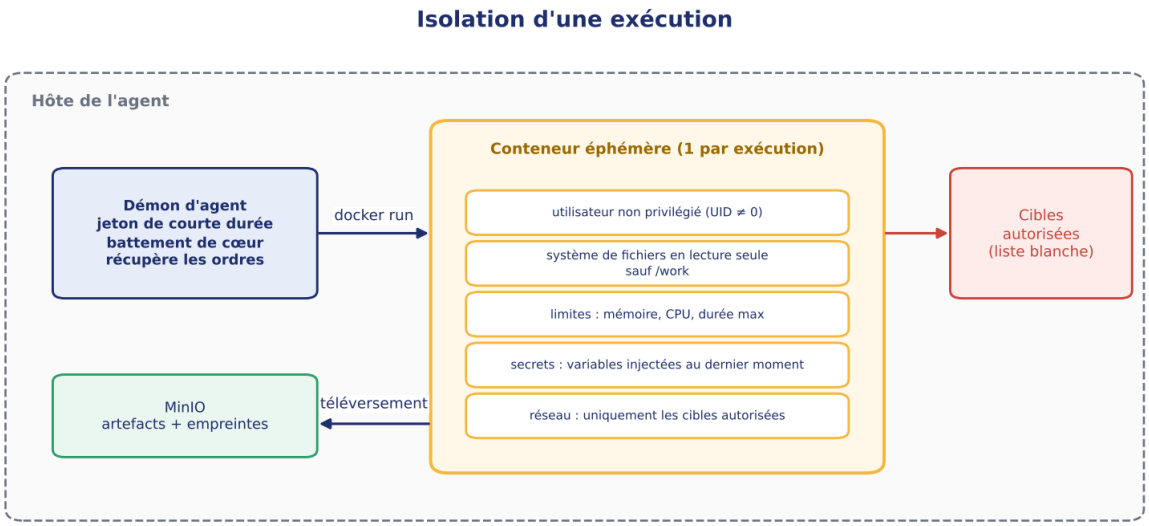


Figure 10.1. Isolation d'une exécution.

Principes

- Moindre privilège : sept rôles, droits par projet vérifiés côté serveur.
- Défense en profondeur : identité, rôle, cible autorisée, plage horaire, quota, arrêt d'urgence, verrou.
- Aucun port entrant sur les agents ; jetons de courte durée, révocables.
- Secrets jamais en base, en journal ou en rapport ; masquage systématique et rotation.
- Tout refus et toute modification sont tracés.

Matrice des droits

ACTION	ADMIN	RESP.	TECH LEAD	DÉV.	QA	EXPL.	AUDIT
Créer projets et équipes	•	•					
Gérer rôles et intégrations	•						
Déposer un script	•		•		•		
Lancer un test autorisé	•		•	•	•		

ACTION	ADMIN	RESP.	TECH LEAD	DÉV.	QA	EXPL.	AUDIT
Annuler une exécution	●		●		●	●	
Définir les seuils			●				
Valider un résultat			●				
Approuver une promotion (lot CI/CD)			●				
Déployer / retour arrière (lot CI/CD)						●	
Arrêt d'urgence	●					●	
Consulter les résultats	●	●	●	●	●	●	●
Consulter et exporter les preuves	●		●				●

Matrice proposée, ajustable en atelier. Une approbation à deux niveaux peut ajouter le responsable entreprise.

Analyse des menaces

MENACE	SCÉNARIO	MESURE
Usurpation	Appel de webhook falsifié	Signature HMAC, horodatage, identifiant d'événement
Falsification	Script modifié entre dépôt et exécution	Empreinte vérifiée avant exécution ; versions immuables
Répudiation	Nier avoir lancé ou approuvé	Journal d'audit en écriture seule
Divulgation	Secret dans un journal ou un rapport	Masquage, test de fuite en pipeline, références seules en base
Déni de service	Charge sur une cible non prévue	Liste blanche, contrôle de l'adresse résolue, quotas, arrêt d'urgence
Élévation de privilèges	Script qui s'échappe de son conteneur	Conteneur non privilégié, système de fichiers en lecture seule, capacités retirées, pas d'accès au socket Docker
Chaîne d'approvisionnement	Image ou dépendance compromise	Analyse des vulnérabilités, versions figées, liste de composants ; signature d'artefacts en évolution

Isolation des organisations

Le cahier des charges demande, en cible, une isolation logique des organisations avec filtrage par tenant côté serveur. La première version gère une organisation, avec projets et équipes cloisonnés ; le modèle de données porte déjà l'identifiant d'organisation, ce qui permet d'ajouter l'isolation multi-organisations sans refonte (évolution).

Chaîne d'approvisionnement et tests de sécurité

- Images scannées, dépendances vérifiées, liste de composants (SBOM) et signature d'artefacts : cible d'évolution.
- Analyses de sécurité dynamiques (DAST) uniquement sur les environnements et cibles autorisés par écrit.
- Conservation et suppression des journaux selon les politiques validées avec JS2E.
- Plan de réponse aux incidents : isoler, révoquer les jetons, conserver les preuves, prévenir JS2E.

11

Fiabilité, continuité et reprise

EN BREF Une panne arrive toujours. La question est de savoir ce que la plateforme fait quand elle survient : elle s'arrête proprement, le dit, et ne tente jamais de « réparer » en relançant une charge.

PANNE	EFFET	COMPORTEMENT DE LA PLATEFORME
Agent perdu en cours de test	Test interrompu	Silence détecté ; exécution indéterminée ; alerte ; verrou relâché ; pas de relance
Orchestrateur redémarré	Ordres en cours incertains	Reprise depuis l'état en base ; exécutions douteuses indéterminées
Redis indisponible	File inaccessible	Nouvelles demandes refusées proprement ; rien de perdu en base
PostgreSQL indisponible	Plateforme inutilisable	Arrêt propre ; restauration ou bascule sur la réplique
MinIO indisponible	Artefacts non stockés	Exécution marquée avec artefacts manquants ; nouvelle tentative de téléversement
Cible indisponible	Tests en échec	Résultat consigné ; pas de relance automatique de la charge
Générateur saturé	Mesures faussées	Résultat marqué non fiable
Échec de déploiement	Version incohérente	Retour arrière selon la politique ; incident ouvert (lot CI/CD)

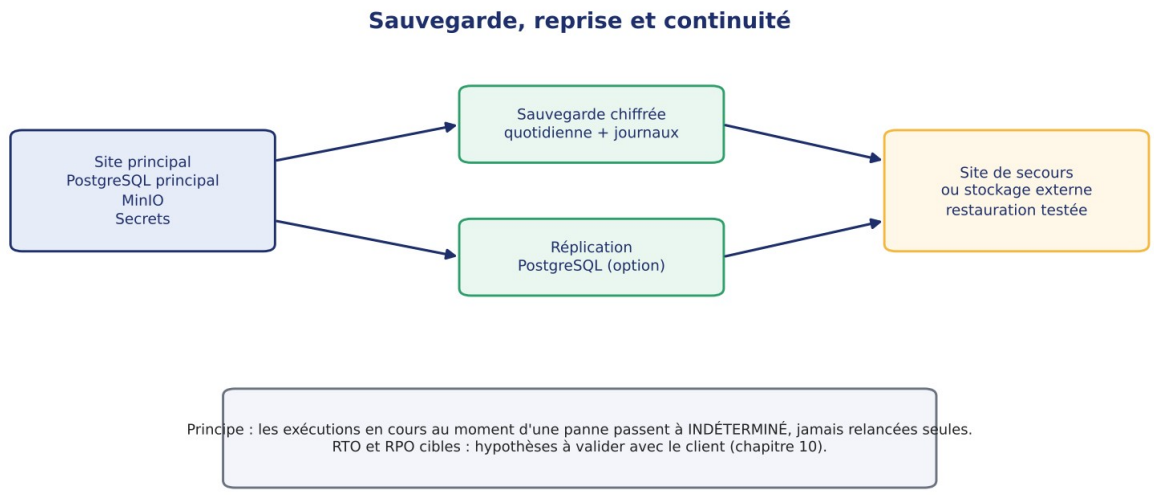


Figure 11.1. Sauvegarde, reprise et continuité.

ÉLÉMENT	FRÉQUENCE PROPOSÉE	RESTAURATION
PostgreSQL	Quotidienne chiffrée, journaux en continu si réplication	Testée en recette, procédure écrite

ÉLÉMENT	FRÉQUENCE PROPOSÉE	RESTAURATION
MinIO	Quotidienne ou réplication	Restauration par préfixe d'exécution
Configuration et déploiement	Versionnée dans le dépôt	Redéploiement par pipeline
Secrets	Selon la politique du coffre	Procédure du coffre

Objectifs de reprise proposés : durée de reprise de huit heures, perte de données maximale de vingt-quatre heures. Hypothèses à valider avec JS2E.

12

Exploitation et supervision

EN BREF Exploiter, c'est voir ce qui se passe, être alerté à temps et savoir quoi faire. La plateforme expose ses propres indicateurs et ceux de la livraison.

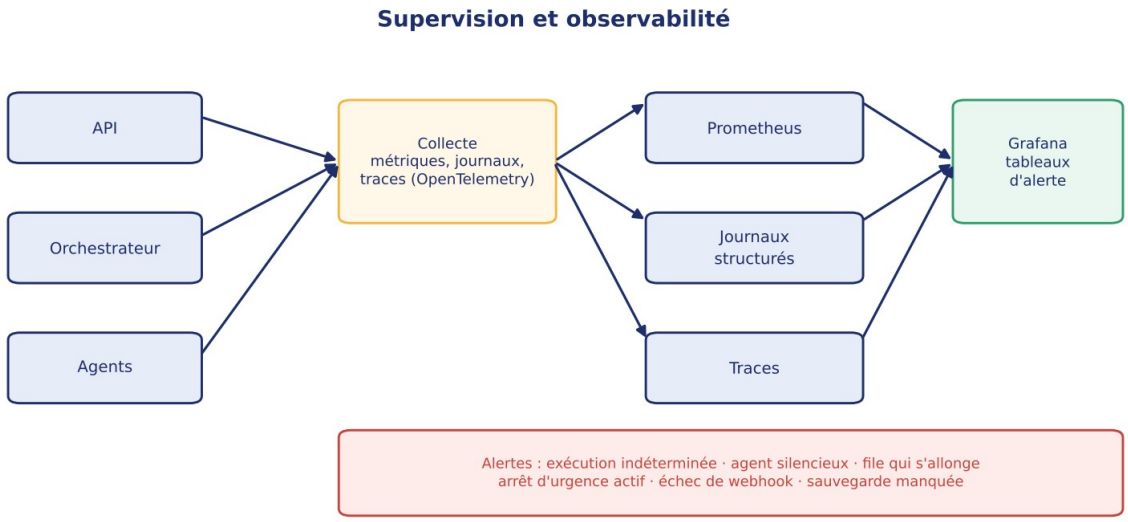


Figure 12.1. Chaîne de supervision.

MÉTRIQUE	TYPE	ALERTE
orchestra_runs_total{state}	compteur	Pic d'erreurs ou d'indéterminées
orchestra_queue_wait_seconds	histogramme	Attente supérieure au délai convenu
orchestra_agent_heartbeat_age_seconds	jauge	Supérieure à trois intervalles
orchestra_generator_saturation_total	compteur	Information : résultats non fiables
orchestra_webhook_failures_total	compteur	Échecs répétés de livraison
orchestra_killswitch_active	jauge	Alerte systématique

Indicateurs de livraison

Le cahier des charges demande de mesurer les temps de pipeline, le taux de réussite, le délai de réparation et la fréquence de livraison. La plateforme calcule ces indicateurs à partir de l'historique des releases et des déploiements.

INDICATEUR	CALCUL
Fréquence de livraison	Nombre de déploiements réussis par période et par application
Délai de livraison	Temps entre le commit et le déploiement réussi
Taux d'échec	Déploiements en échec ou en retour arrière divisés par le total

INDICATEUR	CALCUL
Délai de réparation	Temps entre un échec et le retour à un état conforme
Taux de réussite des tests	Tests réussis divisés par les tests exécutés, avec part d'instables

Journaux et procédures

- Journaux structurés en JSON, identifiant de corrélation par exécution, repris par l'API, l'orchestrateur et l'agent.
- Aucun secret, aucun en-tête d'autorisation, aucune donnée de test sensible dans les journaux.
- Centralisation vers la solution de JS2E si elle existe.

SITUATION	ACTION
Exécution indéterminée	Vérifier la cible, clore ou relancer manuellement, consigner la décision
Arrêt d'urgence	Activer le drapeau, vérifier l'arrêt des conteneurs, prévenir les équipes, lever après analyse
Agent silencieux	Vérifier l'hôte, révoquer le jeton si nécessaire, réenrôler
Échec de déploiement	Appliquer la politique de retour arrière, ouvrir l'incident, informer l'approbateur
Montée de version	Sauvegarde, migration de base, déploiement des images, test de fumée
Incident de sécurité	Isoler, révoquer les jetons, conserver les preuves, prévenir JS2E

13

Performance et passage à l'échelle

EN BREF Quand JS2E utilisera la plateforme davantage, que faut-il ajouter ? Ici, le point sensible n'est pas l'interface mais le poste qui génère la charge.

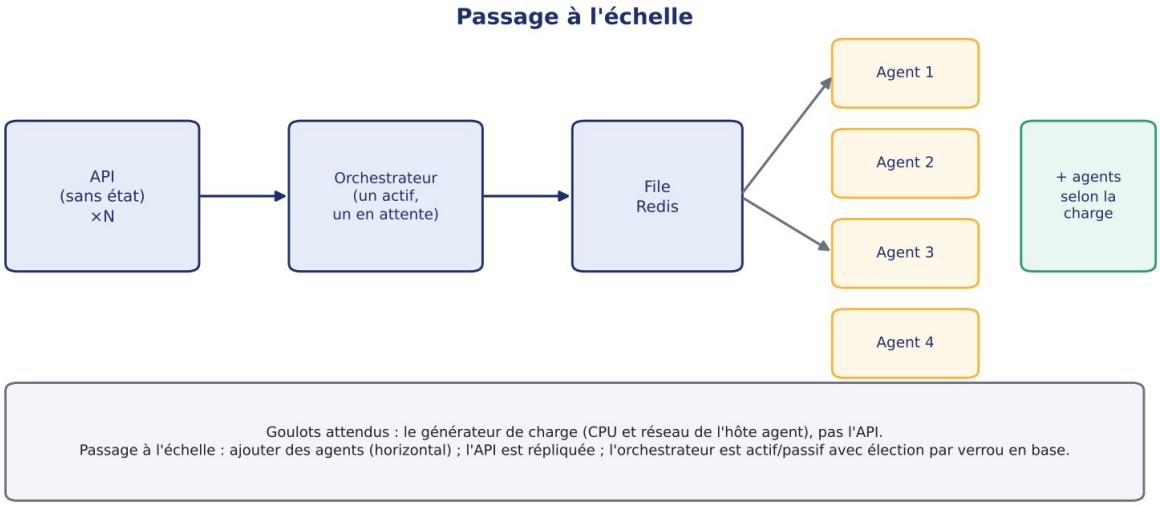


Figure 13.1. Passage à l'échelle par ajout d'agents.

ZONE	RISQUE	PARADE
Générateur de charge	Saturation processeur ou réseau	Surveillance, plusieurs générateurs, hôtes dédiés
API	Pics d'appels de pipelines	Réplicas sans état, limitation de débit par client
Base de données	Volume d'événements et de métriques	Index, agrégation, purge selon la conservation
Lecture du JTL	Fichiers volumineux	Lecture en flux, agrégation par fenêtre
Stockage d'artefacts	Croissance	Durées de conservation, nettoyage tracé

- Ajouter des agents pour exécuter davantage en parallèle ; les quotas protègent les cibles.
- API répliquée derrière le reverse proxy ; orchestrateur actif avec un second en attente, élection par verrou en base.
- Une file par connecteur pour isoler un moteur lent des autres.
- Aucune optimisation prématurée : le dimensionnement de départ est validé par essai sur l'application pilote.

14

Réalisation par lots et recette

EN BREF La solution se construit par étapes. Chaque lot se termine par une condition de sortie que JS2E peut vérifier avant de passer au suivant.

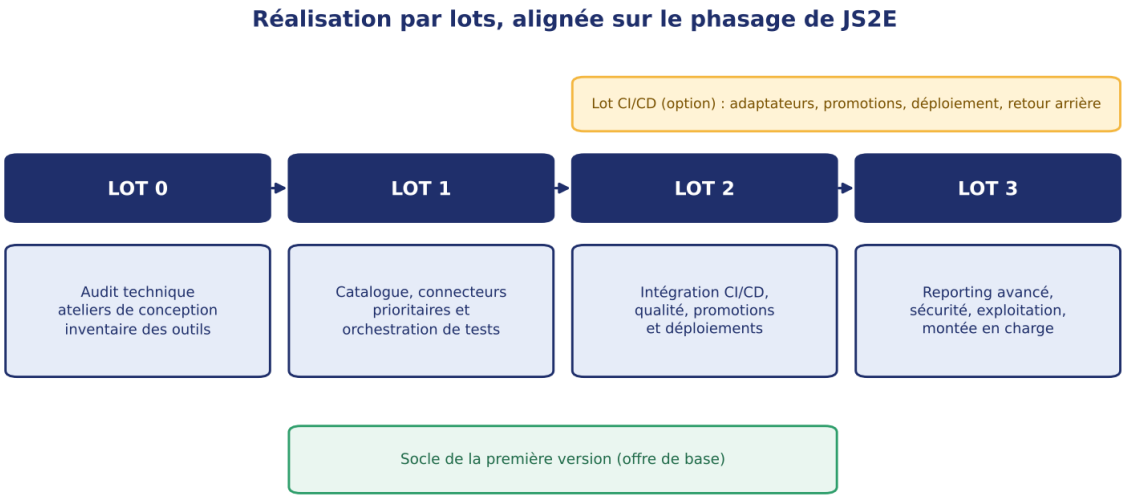


Figure 14.1. Lots de réalisation, alignés sur le phasage proposé par JS2E.

LOT	CONTENU	CONDITION DE SORTIE
Lot 0	Audit technique, ateliers de conception, inventaire des outils, application pilote, risques	Architecture validée, backlog priorisé, accès disponibles
Lot 1	Référentiel, catalogue de scripts, connecteurs JMeter et Selenium, orchestration, verdicts, rapports	Parcours testeur démontré de bout en bout sur l'application pilote
Lot 2 (option CI/CD)	Adaptateurs CI/CD, promotions, approbations, déploiement en recette, retour arrière	Recette fonctionnelle et sécurité validées
Lot 3	Reporting avancé, durcissement de sécurité, exploitation, montée en charge	Reprise testée, indicateurs en service, documentation remise

Matrice de priorisation du cahier des charges

LOT DU CAHIER DES CHARGES	OBJECTIF	POSITION DANS CETTE ARCHITECTURE
MVP	Connexion Git + exécution de test + déploiement en recette	Exécution de test : première version. Connexion Git et déploiement : lot CI/CD
V1	Approbations, rapports, audit, retour arrière	Rapports et audit : première version. Approbations et retour arrière : lot CI/CD
V2	Connecteurs et infrastructures complémentaires	Évolution : Playwright, tests d'API, autres moteurs

LOT DU CAHIER DES CHARGES	OBJECTIF	POSITION DANS CETTE ARCHITECTURE
V3	IA, optimisation, industrialisation avancée	Hors première version

Le cahier des charges place le déploiement en recette dans son MVP. Notre offre le traite en lot CI/CD optionnel : l'arbitrage est à faire en atelier. L'architecture le prévoit dès le départ, ce qui évite toute refonte.

Stratégie de test de la plateforme

FAMILLE	CE QU'ELLE PROUVE
Unitaires	Transitions, quotas, calcul de verdict, détection de saturation
Intégration	Base, file, stockage réels ; idempotence ; verrous
Pannes	Agent tué, base coupée, redémarrage de l'orchestrateur : aucune double exécution
Sécurité	Droits, webhooks, cibles, fuite de secrets
Bout en bout	Parcours testeur complet ; parcours de promotion et retour arrière (lot CI/CD)
Restauration	Sauvegarde puis restauration complète

Livrables

- Ce dossier d'architecture et la cartographie des flux.
- Backlog priorisé avec histoires d'utilisateur et critères d'acceptation.
- Scripts d'installation, description de l'infrastructure, modèles de pipelines.
- Documentation d'exploitation, de sauvegarde, de restauration et de retour arrière.
- Plan de tests, procès-verbal de recette, transfert de compétences.

15

Décisions d'architecture

EN BREF Une décision d'architecture est un choix que l'on a pesé contre une alternative. Les noter permet à JS2E de comprendre, de contester et de faire évoluer.

RÉF.	DÉCISION	RAISON	ALTERNATIVE ÉCARTÉE
AD1	Python 3.12 et FastAPI pour l'API et l'orchestrateur	Productivité, typage, écosystème d'outils de test	Java : plus lourd pour ce volume
AD2	Une machine à états unique, propriétaire de l'état	Éviter les états incohérents	États modifiés par plusieurs composants
AD3	Verdict en trois dimensions	Un test terminé n'est pas forcément bon ni fiable	Un seul statut
AD4	Agents sans port entrant, conteneurs éphémères	Réduire la surface d'attaque	Agents exposés en service
AD5	Aucun relancement automatique après incident de charge	Éviter de doubler la charge sur une cible sensible	Relance avec quota
AD6	PostgreSQL avec verrous consultatifs pour les cibles	Cohérence transactionnelle sans composant de plus	Verrou distribué dans Redis
AD7	Contrat de connecteur unique	Ajouter un moteur sans toucher au cœur	Intégrations spécifiques par moteur
AD8	Interface sans règle métier	Un seul lieu de vérité côté serveur	Logique dupliquée dans le navigateur
AD9	Docker sur site, compose en première version	Simplicité d'exploitation	Kubernetes dès le départ
AD10	Selenium d'abord, Playwright ensuite derrière le même contrat	Choix initial de JS2E, extension prévue	Les deux dès le début
AD11	Orchestrer le pipeline existant	Pas de migration imposée à JS2E	Remplacer les pipelines
AD12	Approbation humaine avant production	Décision critique laissée à l'humain	Promotion entièrement automatique

16

Correspondance avec le cahier des charges JS2E

EN BREF Ce tableau permet à JS2E de vérifier, point par point, que son cahier des charges est couvert, et où.

SECTION DU CAHIER DES CHARGES	RÉPONSE	CHAPITRE	STATUT
Périmètre prioritaire : référentiel, import de scénarios	Référentiel, catalogue de scripts, connecteurs	4, 6, 7	Couvert
Déclenchement manuel, planifié, webhook, après pipeline	API, planificateur, webhooks signés	6, 8	Couvert
Exécution parallèle, quotas, isolation	Agents, conteneurs, quotas	6, 10, 13	Couvert
Rapports consolidés et tendances	Résultats normalisés, rapports, comparaisons	4, 7	Couvert
Orchestration CI/CD et promotion entre environnements	Module Livraison	5	Lot CI/CD (option)
Quality gates bloquants ou informatifs	Seuils, verdict	4	Couvert
Traçabilité et preuves exportables	Audit, historique, export	7, 10	Couvert
Connecteurs JMeter, Selenium/Playwright, API	Trois connecteurs ; Playwright après qualification	6	Couvert, Playwright en évolution
Adaptateurs GitHub, Jenkins, GitLab, Azure DevOps	Adaptateurs à qualifier	8	Lot CI/CD
Déploiement SSH/agent, Docker, Kubernetes	Agents de déploiement	5	Lot CI/CD, selon inventaire
Webhooks signés, REST, JSON, JUnit, Prometheus	Mécanismes d'échange	8	Couvert
Acteurs et habilitations (sept rôles)	Matrice des droits	4, 10	Couvert
Parcours de bout en bout en dix étapes	Tableau de couverture	4, 5	Première version et lot CI/CD
Exigences transverses : audit, secrets, RBAC, notifications, API	Chapitres sécurité, intégration, API	6, 8, 10	Couvert
SSO/OIDC, MFA administrateurs	Keycloak, MFA	8, 10	Couvert
Isolation logique des organisations	Modèle prêt, isolation ajoutée plus tard	10	Évolution
SBOM et signature d'artefacts	Cible d'évolution	10	Évolution
DAST sur cibles autorisées	Encadré par autorisation écrite	10	Évolution
Architecture logique : front, back, file, base, registre, vault, runners, OpenTelemetry	Composants proposés	6, 9	Couvert
PostgreSQL/Supabase avec RLS	PostgreSQL ; RLS possible	7	À valider
Sauvegarde, restauration testée, rétention	Procédures et objectifs	7, 11	Couvert
Audit de découverte obligatoire	Lot 0	14	Couvert
Phasage en lots 0 à 3	Alignement des lots	14	Couvert
Livrables projet	Liste des livrables	14	Couvert

SECTION DU CAHIER DES CHARGES	RÉPONSE	CHAPITRE	STATUT
Risques et mesures	Chapitre 17	17	Couvert

17

Risques et points ouverts

Risques d'architecture

RISQUE	MESURE
Intégration hétérogène des outils de JS2E	Essai de faisabilité sur les outils réels pendant le lot 0
Accès aux serveurs de déploiement	Agents isolés, identités de courte durée, flux audités
Tests de charge sur des environnements sensibles	Autorisations écrites, quotas, plages horaires, arrêt d'urgence
Secrets	Jamais journalisés : masquage dans les journaux et rapports
Retour arrière	Distinguer l'image applicative des migrations irréversibles
Générateur sous-dimensionné	Essai de charge sur l'application pilote avant de figer le dimensionnement
Dérive de périmètre	Gouvernance du backlog, jalons et validation de chaque lot

Points à trancher en atelier avec JS2E

N°	QUESTION	IMPACT
1	Quels outils Git, CI/CD, tickets, runners et gestion des secrets sont en place ?	Chapitres 5, 8
2	Quelles applications (J2EE et autres), environnements et dépendances ?	Chapitres 4, 6
3	Quels types de tests, volumes, données de test, contraintes réseau, fenêtres de tir ?	Chapitres 6, 9, 13
4	Quel hébergement, quelles règles réseau, quelle identité (SSO) ?	Chapitres 9, 10
5	Résidence des données, durées de conservation, sauvegarde ?	Chapitres 7, 11
6	Le déploiement et le retour arrière sont-ils dans la première livraison ? Sur quelles cibles ?	Chapitres 5, 14
7	Combien de projets, d'équipes, d'utilisateurs, d'exécutions simultanées ?	Chapitres 3, 13
8	Quels indicateurs de réussite et quel périmètre du pilote avec le sponsor ?	Chapitre 14
9	Objectifs de disponibilité, de durée de reprise et de perte de données acceptée ?	Chapitre 11
10	Isolation entre organisations : nécessaire dès le départ ?	Chapitre 10

Glossaire

TERME	SENS
Agent	Programme qui exécute les ordres de la plateforme dans un conteneur isolé
API	Interface par laquelle d'autres logiciels dialoguent avec la plateforme
Approbation	Décision humaine tracée avant une promotion ou un déploiement
Cible	Application ou environnement autorisé à recevoir un test
Conteneur	Boîte logicielle isolée où s'exécute un test
Connecteur	Module qui pilote un outil de test selon un contrat unique
Idempotence	Rejouer la même demande ne crée pas une seconde exécution
JTL	Fichier de résultats de JMeter

TERME	SENS
Kill switch	Arrêt d'urgence global ou par cible
MFA	Authentification à plusieurs facteurs
OIDC	Standard d'identité utilisé pour la connexion unique (SSO)
Promotion	Passage d'une version d'un environnement à l'environnement suivant
Quality gate	Règle de seuils qui produit un verdict lisible par un pipeline
Release	Version identifiée d'une application, liée à un commit et à un build
Retour arrière	Redéploiement de la version précédente après un échec
RTO / RPO	Durée maximale de reprise / perte de données maximale acceptée
Verrou de cible	Un seul test de charge à la fois sur une même cible
Webhook	Appel automatique d'un système à un autre, ici signé pour être authentifié